



SINGAPORE UNIVERSITY OF
TECHNOLOGY AND DESIGN

50.039 Theory and Practice of Deep Learning

Group Member Name	Student ID	Contribution
Chadrick Liang	1007998	2, 4, 5, 6
Iris Chan Hui Qi	1007985	1, 2, 3, 6
Tan Yi Jie Sean	1007775	1, 4, 5, 6

Github repo:

https://github.com/Bearson678/fake-job-posting_DL_Project.git

Table of Contents

1. Introduction.....	4
1.1 Dataset.....	4
1.2 Machine Learning Problem.....	4
1.2.1. Task.....	4
1.2.2. Inputs / Outputs.....	4
1.2.3. Model Overview.....	4
1.2.4. Loss Function.....	6
1.2.5 Installation & Fresh Run.....	6
2. Exploratory Data Analysis (EDA).....	7
2.1 Data Cleaning.....	7
2.2 Univariate Analysis.....	7
2.2.1. Numerical Column Analysis.....	7
2.2.2. Categorical Column Analysis.....	9
2.2.3. Free Text Column Analysis.....	13
2.3 Feature Engineering.....	15
2.3.1.Text Length Analysis.....	15
2.3.2. Finalized Set of Features.....	16
2.4 Prepping Dataset.....	17
3. Embeddings.....	17
3.1 Data Preprocessing.....	17
3.2 FastText vs CBoW.....	17
2.2.2. Hyperparameter Tuning.....	18
4. Model Architecture.....	20
4.1 Model Blocks.....	20
4.1.1. BiGRU.....	20
4.1.2. Multi-head Attention Pooling.....	21
4.1.3. Numerical Block.....	22
4.2 Model Implementation.....	22
4.2.1. Block Instantiation.....	23
4.2.2. Forward method.....	24
4.2.3 Fit method.....	25
4.2.4 Evaluation.....	27
5. Model Training.....	30
5.1 Data Preprocessing.....	30
5.1.1. Train Test Validation Split.....	30
5.1.2. Non-binary Value Standardisation.....	30
5.1.3. Tokenization, Encoding, Embedding Matrix.....	30
5.1.3. Dataset Dataloader Instantiation.....	30
5.2 Model Training.....	32

5.2.1. Model Instantiation.....	32
5.2.2. Training Loop.....	33
5.2.3. Test Set Evaluation.....	33
5.2.4. Result Visualizations.....	34
5.2.5. Hyperparameter Tuning.....	38
5.2.6. Load Finalized Model.....	42
6. Model Evaluation.....	43
6.1 Model Experimentation.....	43
6.1.1. Distilbert Model.....	43
6.1.2. Numerical and NLP Branch Evaluation.....	44
6.1.3. Loss Function Experimentation.....	46
6.2 Comparison against similar Kaggle Models.....	46
6.2.1. Bi-LSTM Architecture.....	46
6.2.2. BERT Architecture.....	47
6.3 Malfunction.....	48
6.3.1. White Box Adversarial Attack.....	48

1. Introduction

Online job boards such as LinkedIn and Indeed have made job seeking more accessible and convenient, but also have enabled the rise of fraudulent job postings designed to scam applicants or steal personal information. Detecting fake job listings is therefore crucial to protect users and maintain trust in these online systems.

1.1 Dataset

<https://www.kaggle.com/datasets/shivamb/real-or-fake-fake-jobposting-prediction/data>

1.2 Machine Learning Problem

1.2.1. Task

This dataset contains 18k job postings, where around 800 are fake. This leads to a class imbalance of **95.5% real job postings** and **4.5% fake postings**. The task is a supervised binary classification task to detect anomalies based on a series of details given by the posting. As fake job postings are rare, and job seekers may suffer financial and emotional harm from scams if not detected.

1.2.2. Inputs / Outputs

It contains 17 input features (**job_id**, **title**, **location**, **department**, **salary_range**, **company_profile**, **description**, **requirements**, **benefits**, **telecommuting**, **has_company_logo**, **has_questions**, **employment_type**, **required_experience**, **required_education**, **industry**, **function**) and 1 binary classification output (**fraudulent**): 0 if real, 1 if fake. The input features are either boolean (1,0) or text, and may have null values.

1.2.3. Model Overview

We use a recurrent neural network (RNN) with 2 parallel branches, an NLP branch that focuses on the free text columns in the dataset, as well as a numerical branch to focus on boolean values, which would be concatenated to form the final output.

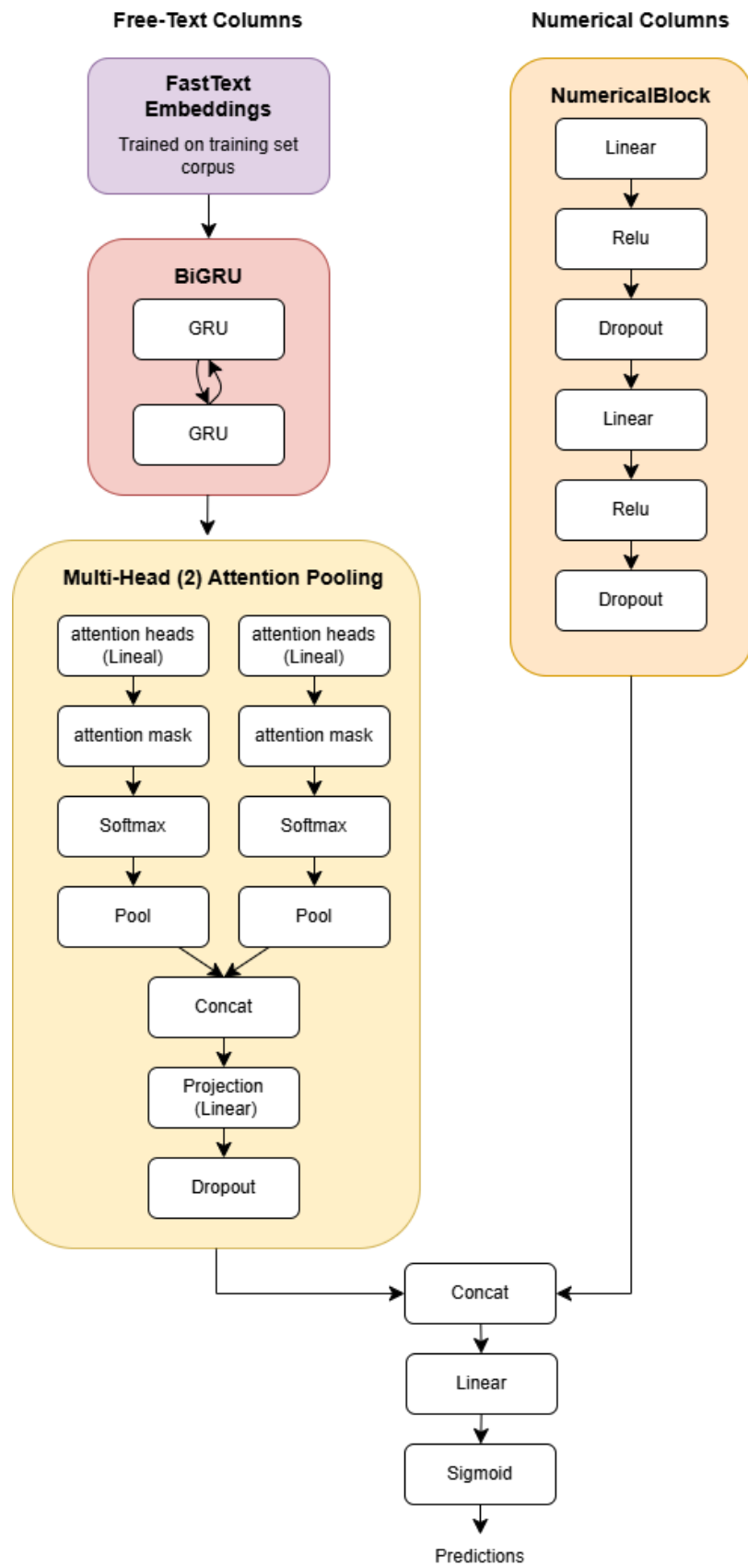


Figure 1: Overview of the model architecture

1.2.4. Loss Function

We use sigmoid focus loss as it is a loss function designed to handle class imbalances by using parameters like gamma to minimize the loss contributions from confident and correct classifications. Thus, the majority of the loss accumulated is from confident misclassifications and model guessings. Whereas alpha is a weight that is applied to loss function to balance positive and negative classes, which addresses the class imbalance issue that we have witnessed in the dataset.

1.2.5 Installation & Fresh Run

To set up the project, create a Python virtual environment and install the required dependencies from 'requirements.txt' via the terminal commands:

```
py -3.10 -m venv .venv310
.\.venv310\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
jupyter lab
```

Figure 2: Code to create the virtual environment and install dependencies

To train the model from scratch, ensure that “**fake_job_postings_ALL.csv**” is in the “**/data/clean**” directory.

To download the clean weights, go to

<https://drive.google.com/drive/folders/1buxqOhkYUN2XV3OI7jawNvOFpSrOKMNH?usp=sharing>. and download both files, inserting it in the root folder of the program.

Under **Model_training.ipynb**, run all of the following cells from “**Dataset Loading**” to “**Model Training**”.

Below are the markdown cell sections and their functions in notebook order:

- **Dataset Loading**: Makes all necessary imports, sets torch seed and loads in cleaned dataset.
- **Train-Test-Validation Split**: Splits up the dataset and converts value type for tensor conversion.
- **Non-binary value Standardisation**: Standardises non-binary numeric columns
- **Loading of fine-tuned FastText model**: loads the FastText model to extract embedding matrix
- **Tokenizing and encoding tokens into numerical values**: provide numerical representations for the text samples and cache them
- **Instantiating Datasets**: Instantiate Dataset and DataLoader class
- **Testing Dataloader** by sampling a batch: ensures that the custom dataset class is working

- **Creating the embedding matrix:** gets the embeddings from the FastText model and prepares it to be copied into the model
- **Model Instantiation:** Instantiates the model
- **Model Training:** Trains the model
- **Hyperparameter Threshold Tuning:** Hyperparameter tuning of the sigmoid threshold value
- **Model Branch Evaluation:** Compares the performance of both the NLP and numeric branch
- **Loss and Accuracy Visualisations:** Provides visual figures of the results from training
- **Confusion Matrix Report Visualisation:** Provides confusion matrices for evaluation metrics such as precision and recall.
- **Hyperparameter GRU/Hidden layer dimension tuning:** Hyperparameter tuning of the model dimension parameters.

Avoid running the “**Hyperparameter GRU/Hidden layer dimension tuning**” unless necessary, this will take quite some time to run and our best track record was 5 hours.

2. Exploratory Data Analysis (EDA)

The raw dataset can be found in **data/raw/fake_job_postings.csv**, and is loaded in through EDA.iypnb, where univariate analysis of categorical, numerical and free text columns occurs. Some data cleaning and feature engineering is done to prep the dataset. The cleaned data is then exported into **data/fake_job_postings_ALL.csv**, with **data/fake_job_postings_nlp.csv** containing all free text columns and **data/fake_job_postings_numeric.csv** containing all numeric columns.

2.1 Data Cleaning

We clean columns with high missing / null values

- **Salary_range** has 84% missing values → we transform **salary_range** into **salary_provided** boolean column
- **Department** has 64.5% missing values → drop column because too sparse to extract meaning
- **Job_id** → drop column as ids have no predictive power over whether selected job is fraudulent

Given that **location** column follows a country, state, city regex, we create 3 additional columns: country, state and city.

2.2 Univariate Analysis

2.2.1. Numerical Column Analysis

These 4 columns contain only boolean values.

	count	mean	std	min	25%	50%	75%	max
telecommuting	17880.0	0.042897	0.202631	0.0	0.0	0.0	0.0	1.0
has_company_logo	17880.0	0.795302	0.403492	0.0	1.0	1.0	1.0	1.0
has_questions	17880.0	0.491723	0.499945	0.0	0.0	0.0	1.0	1.0
fraudulent	17880.0	0.048434	0.214688	0.0	0.0	0.0	0.0	1.0

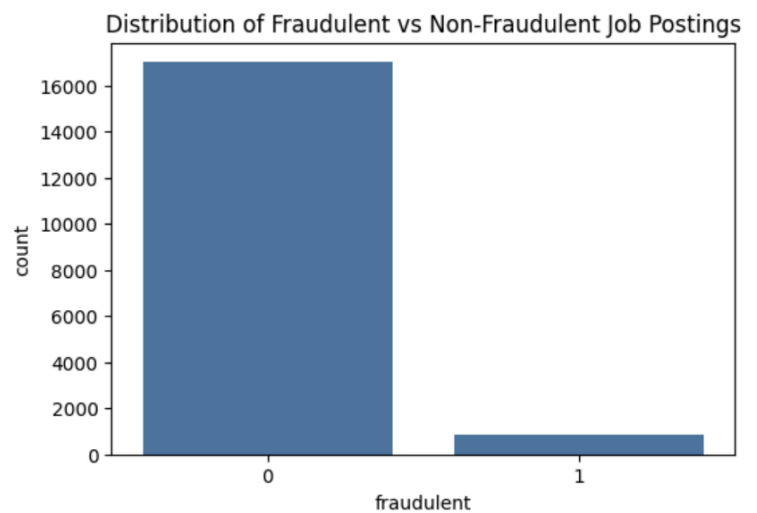
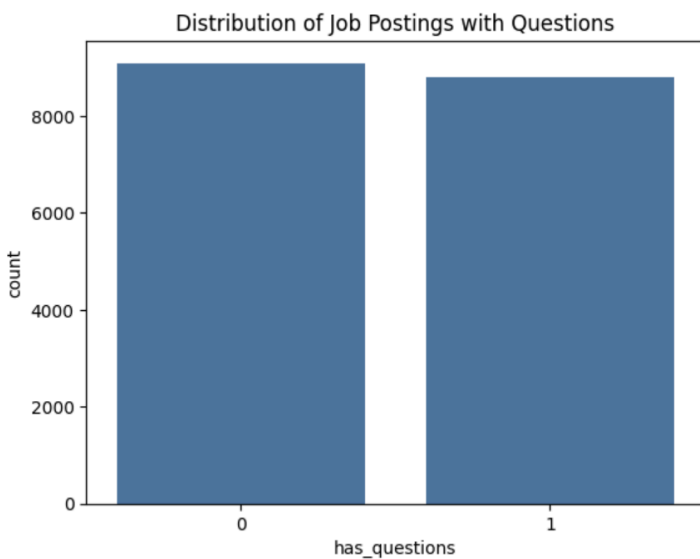
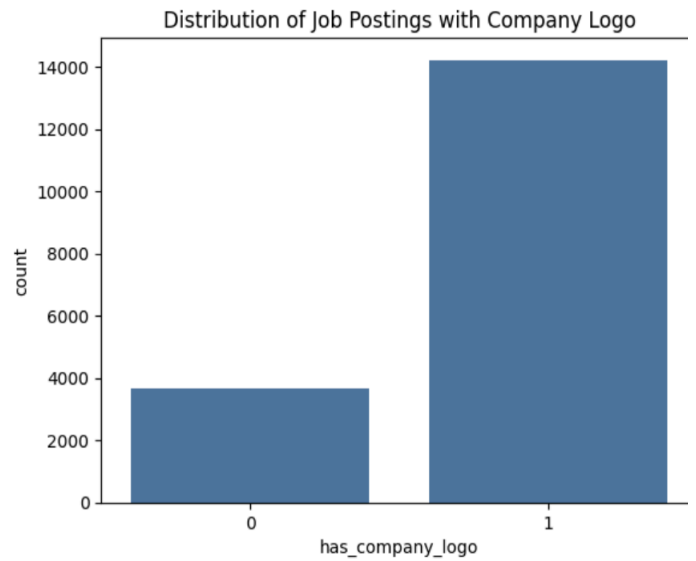
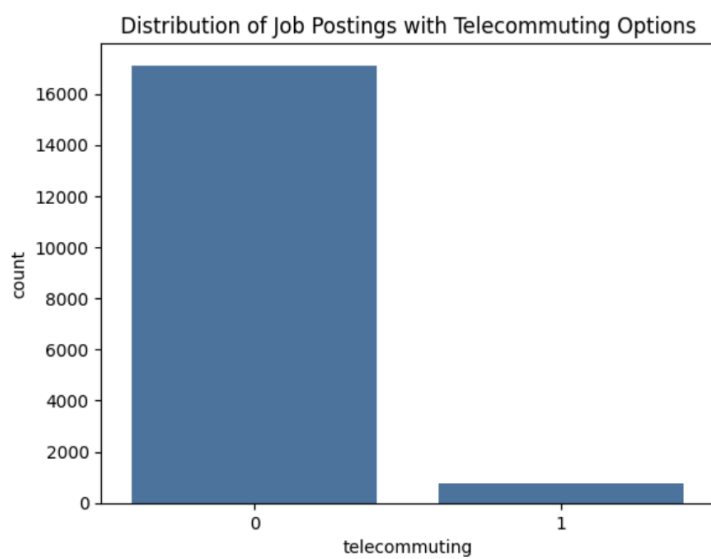


Figure 3: Univariate Results of boolean columns

These results show that:

- Most jobs **do not involve telecommuting** (95.7%)
- Majority of jobs **have a company logo** (79.5%)
- There is an **even split** on whether the job posting contains questions in the description (50.8%)
- Majority of jobs are **not fraudulent** (95.1%)

2.2.2. Categorical Column Analysis

We analyse how different columns in each column lead to different proportions of job postings to be fraudulent or not, comparing **count** and **rate of occurrence**. There are 8 categorical columns, namely **Employment Type, Required Experience, Required Education, Function, Industry, Country, State and City**

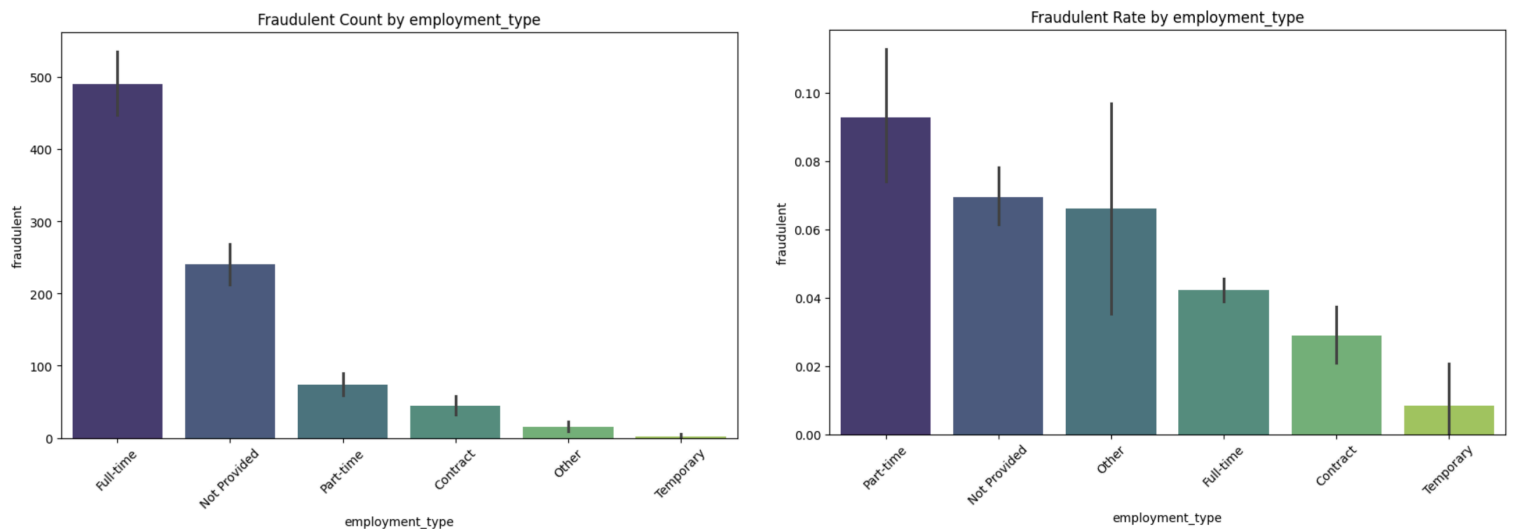


Figure 4: Fraudulent Count and Rate by employment_type

Employment Type

- Majority of fraudulent jobs are for **full time positions**
- However, **part time jobs** have the greatest proportions of fraudulent job postings

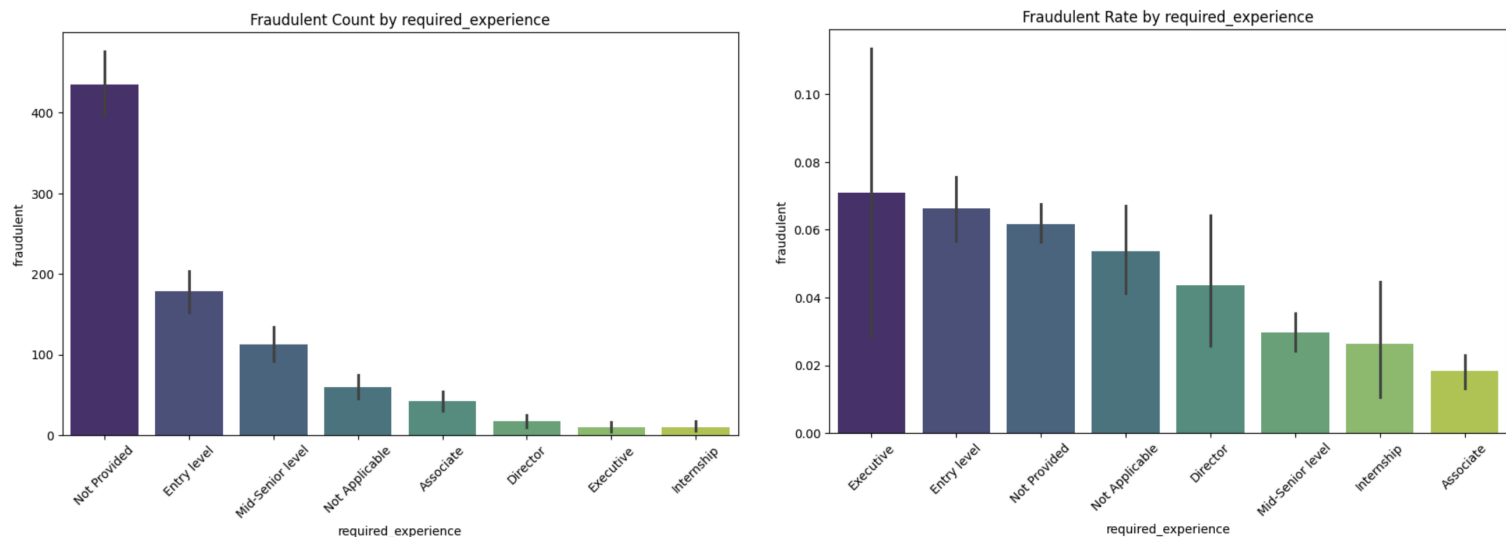


Figure 5: Fraudulent Count and Rate by required_experience

Required Experience

- Majority of fraudulent jobs **have no provided experience**
- However, **executive level jobs** have the greatest proportions of fraudulent job postings

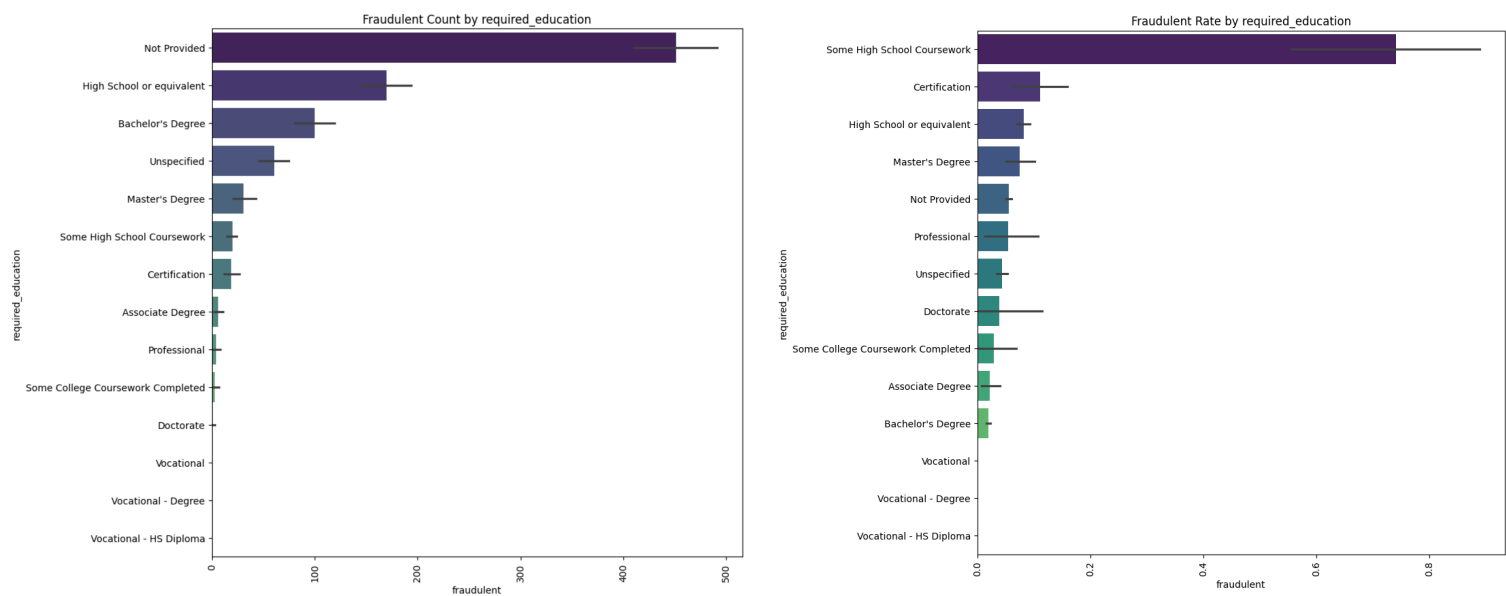


Figure 6: Fraudulent Count and Rate by required_education

Required Education

- Majority of fraudulent jobs **do not have a specified education**
- However, a significant proportion of job postings that only **require high school education** are considered fraudulent

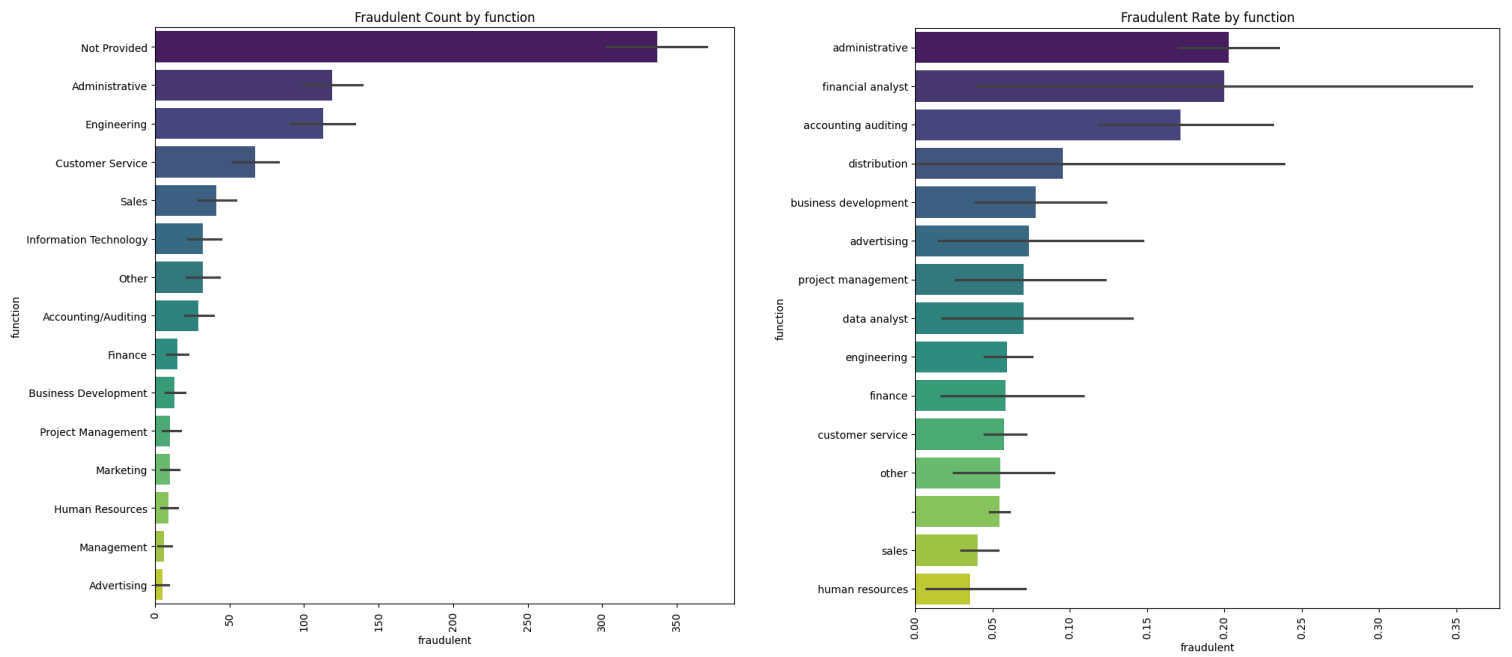


Figure 7: Fraudulent Count and Rate by function

Function

- Majority of fraudulent jobs **have no specified function**
- However, **administrative functions** have the highest proportion of fraudulent job postings

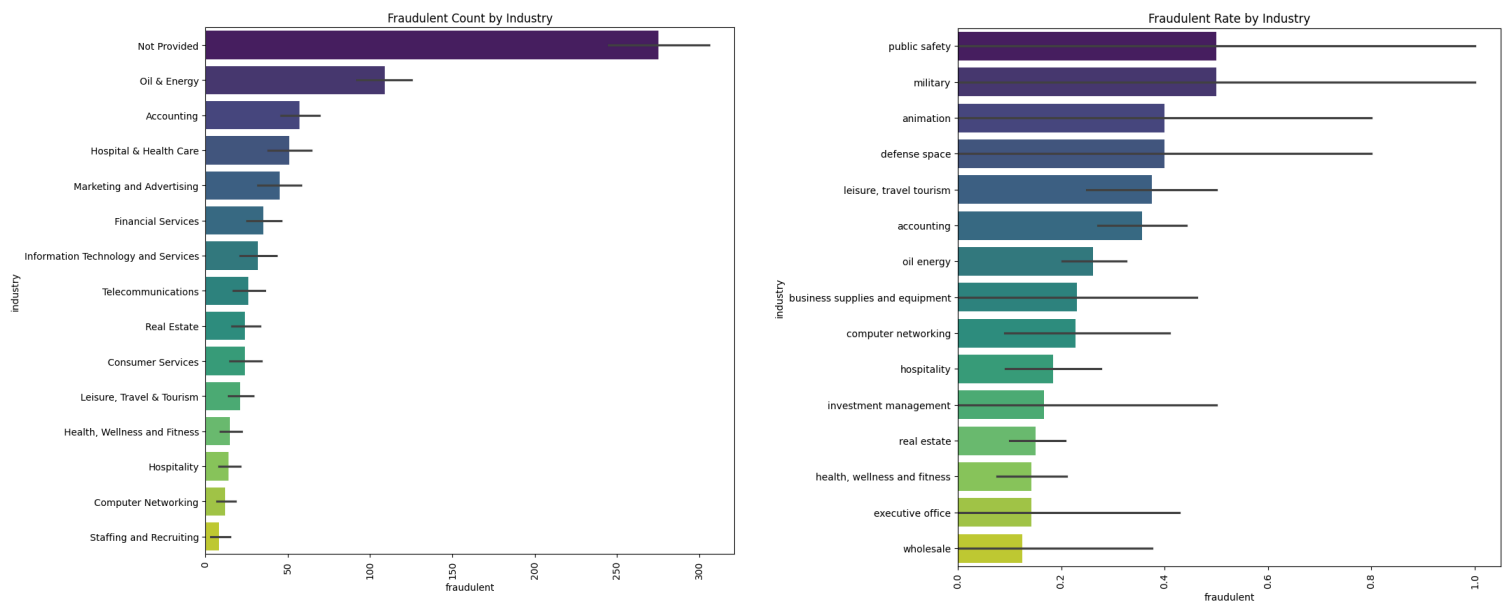


Figure 8: Fraudulent Count and Rate by Industry

Industry

- Majority of fraudulent jobs **have no industry**

- However, **public safety** has the highest proportion of fraudulent jobs

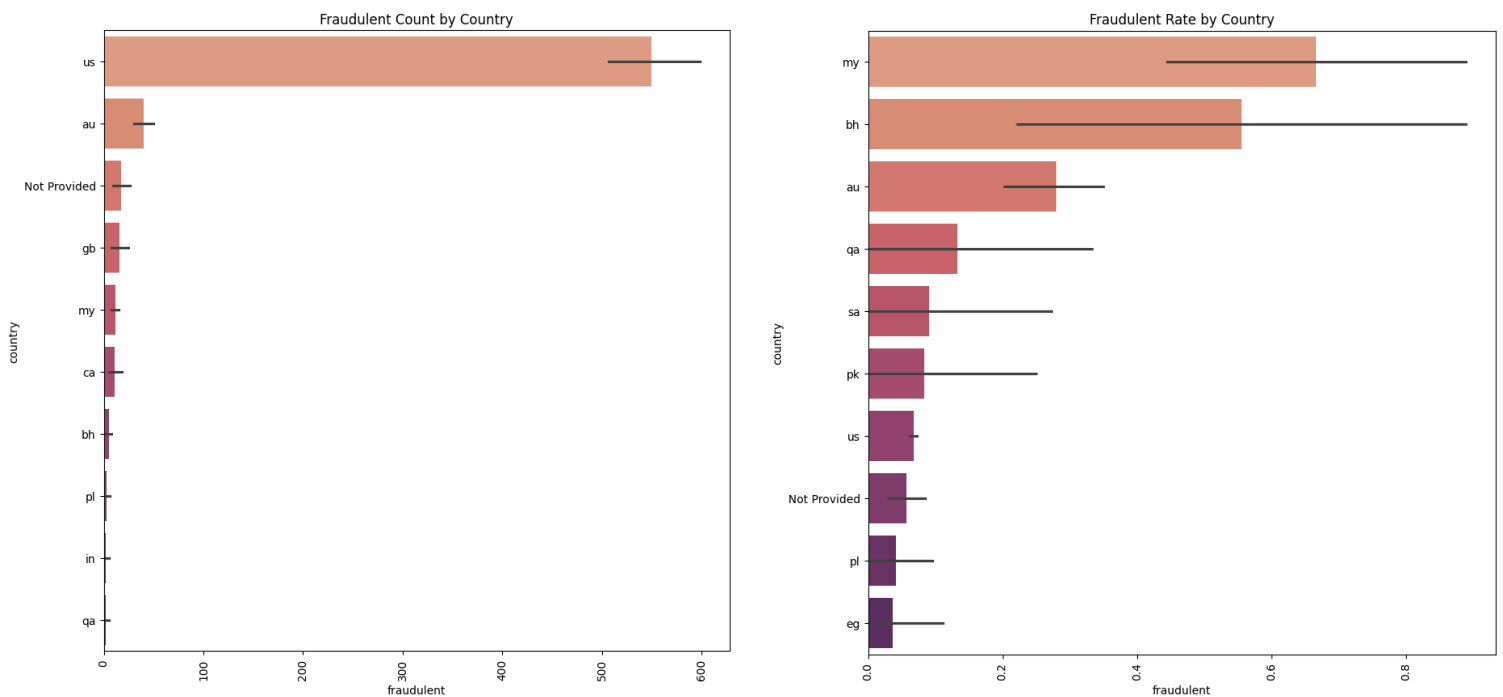


Figure 9: Fraudulent Count and Rate by Country

Country

- Majority of fraudulent jobs come from **US, Australia and Great Britain**
- However, the highest fraudulent rate for fake jobs occurs in **Malaysia, Bahrain, Australia**

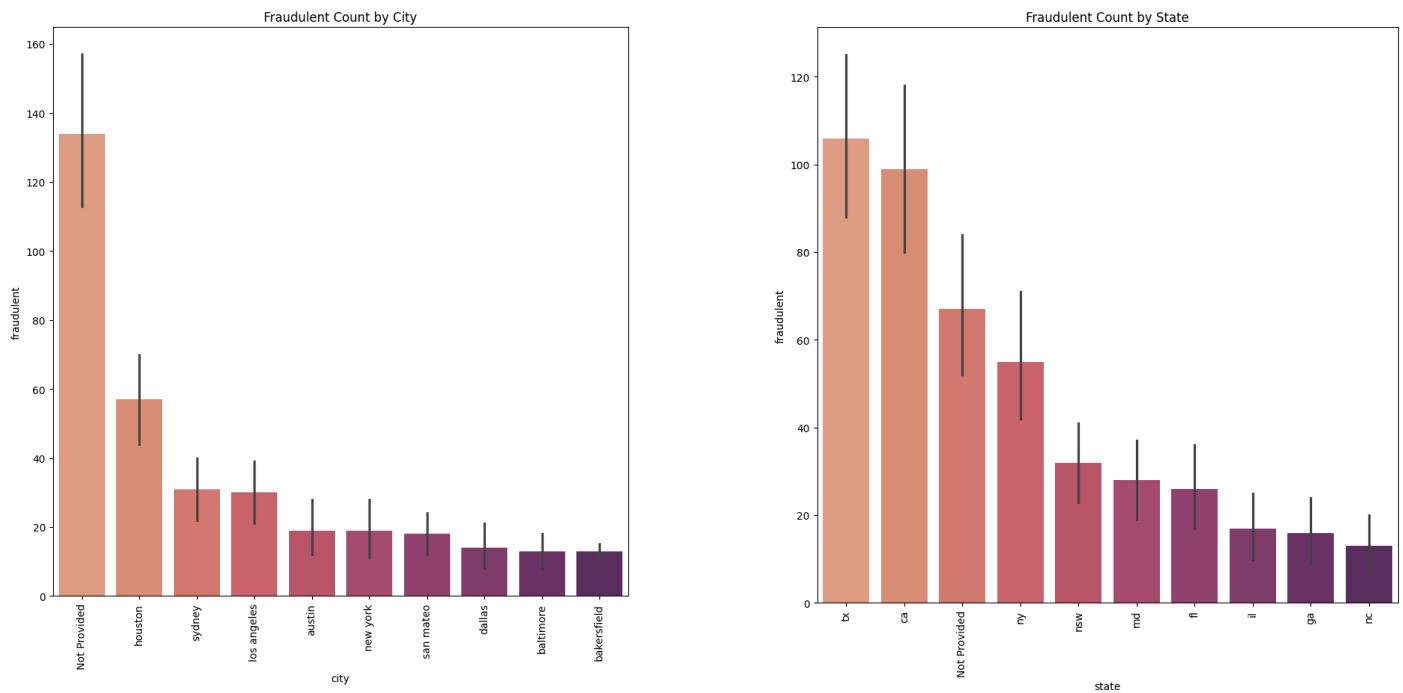


Figure 10: Fraudulent Count and Rate by state

State and City

- Majority of fraudulent jobs **have no specified city**
- Most common states which have fraudulent jobs include **Texas, California, and those with no specified state**

```
# Check if BOTH State and City are missing
df['vague_location'] = (df['state']=='Not Provided') & (df['city'] ==
'Not Provided')
df['vague_location'] = df['vague_location'].astype(int)

# Check the Fraud Rate
vague_rate = df.groupby('vague_location')['fraudulent'].mean()
print("Fraud Rate: Vague (1) vs Specific (0) Locations:")
print(vague_rate)

Fraud Rate: Vague (1) vs Specific (0) Locations:
vague_location
0    0.049303
1    0.056604
```

Figure 11: Proportion of samples having a vague location

We can see that job postings **missing both state and city** have a higher chance of being fake.

We have 4 free text columns: **Job Titles**, **Company Profiles**, **Job Requirements**, **Job Description**. We will preprocess the texts by **implementing custom stopwords** (removing urls) to prevent miscalculation on the importance of seen words. Next, we will use **word clouds** to visualise the spread of words. We will also analyse the **text length** of all text-based columns to find any useful correlation.

```
custom_stopwords = STOPWORDS.union({"will", "want", "able", "amp",
"#URL", "link", "url"})
```

Figure 12: Set of Stopwords used



Figure 13: Word Cloud of Fake and Real Job Titles

Job Titles

- Both real and fake jobs **commonly feature the same titles** such as customer service, engineer
- Data Entry jobs have a higher probability of being fake, while teachers have a higher probability of being real



Figure 14: Word Cloud of Fake and Real Company Profiles

Company Profiles

- Fake profiles often contain words relating to **high paying careers** such as: signing bonus, aptitude staffing, perks

- Real profiles are more **collaborative based** and feature words such as team, client, people



Figure 15: Word Cloud of Fake and Real Job Requirements

Job Requirements

- Both real and fake jobs **commonly feature the same requirements** such as ability, experience
- Thus, it **may not be a good indicator** of whether a job is fake or real

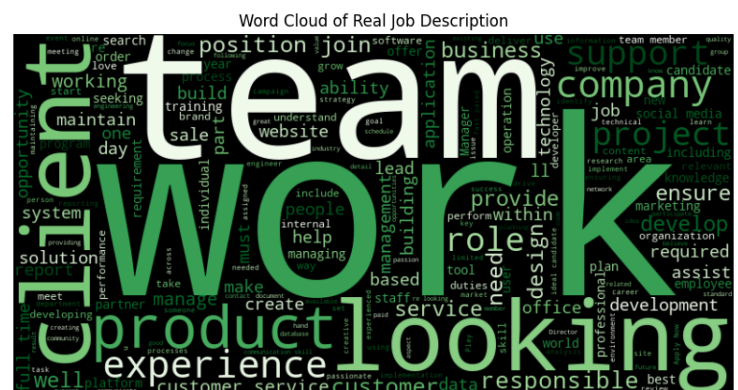


Figure 16: Word Cloud of Fake and Real Job Description

Job Description

- Both real and fake jobs **commonly feature the same description** such as work, team and project

2.3 Feature Engineering

2.3.1. Text Length Analysis

We analyse the **text length** and **word count** of all text-based columns to find any useful correlation.

Company Profile

- Length
 - Real: mean = 640.8 chars, median = 588.0 chars.
 - Fake: mean = 230.9 chars, median = 0.0 chars.
- Word Count
 - Real: mean = 96 words, median = 86 words
 - Fake: mean = 33.1 words, median = 2 words

A median of 0 means over half of fraudulent postings have an **empty company profile**.

Description

- Length
 - Real: mean = 1221.2, median = 1027.0.
 - Fake: mean = 1154.8, median = 844.5.
- Word Count
 - Real: mean = 171.0, median = 147.0.
 - Fake: mean = 158.8, median = 113.5.

Both fake and real postings are **relatively similar** so this is a weak signal ($844.5 / 1027.0 = 0.82$)

Requirements

- Length
 - Real: mean = 599.3, median = 476.5.
 - Fake: mean = 448.0, median = 249.0.
- Word Count
 - Real: mean = 79.3, median = 63.
 - Fake: mean = 58.8, median = 34.

Fraudulent postings provide **roughly half the detail** in requirements ($249.0 / 476.5 = 0.52$).

Benefits

- Length
 - Real: mean = 208.7, median = 47.0.
 - Fake: mean = 212.2, median = 36.0.
- Word Count
 - Real: mean = 30.8, median = 6.
 - Fake: mean = 30.3, median = 5.

Nearly identical and is another weak signal.

We will add **company_profile_len, company_profile_word_count, description_len, description_word_count, requirements_len, requirements_word_count**. This reinforces the pattern that fraudulent postings appear to omit information.

We also investigated the following features:

Missing Count

- Counts the number of empty/unprovided fields across 9 columns (4 text + 5 categorical).
- Fraud rate generally increases with more missing fields: 3.61% at 0 missing, 9.35% at 4, 10.89% at 6, 18.40% at 8.

- Not perfectly linear, certain combinations of missing fields matter more than others but the overall trend is clear.

Total Text Length

- Sum of character lengths across company_profile, description, requirements, benefits
- Fraudulent median: 1,624.5 chars vs legitimate median: 2,530.0 chars ($1,624.5 / 2,530.0 = 0.64$)
- Confirms fraudulent postings provide less textual content.

Is Salary Provided

- Binary flag: 1 if salary_range was provided, 0 if missing.
- Counterintuitively, postings with a salary have a higher fraud rate: $223 / 2,868 \times 100 = 7.78\%$ vs $643 / 15,012 \times 100 = 4.28\%$.
- This goes against the "less info means more fraud" pattern, suggesting fraudsters may use salary as bait.

Has Company Profile

- Binary flag: 1 if company_profile is non-empty, 0 if empty.
- Without profile: $587 / 3,308 \times 100 = 17.74\%$ fraud rate. With profile: $279 / 14,572 \times 100 = 1.91\%$.
- $587 / 866 \times 100 = 67.78\%$ of all fraud cases lack a company profile, the strongest single feature alongside has_company_logo.

2.3.2. Finalized Set of Features

Our finalized set of columns

Text Columns	Title, Country, State, City, Company_Profile, Description, Requirements, Benefits, Employment_Type, Required_Experience, Required_Education, Industry, Function	13
Numeric Columns	Telecommuting, Missing_Count, Total_Text_Len, Company_Profile_Len, Description_Len, Requirements_Len, Benefits_Len, Company_Profile_Word_Count, Description_Word_Count, Requirements_Word_Count, Benefits_Word_Count, Salary_Provided, Has_Company_Profile, Vague_Location, Has_Company_Logo, Has_Questions	16
Non-binary Columns	Total_Text_Length, Company_Profile_Len, Description_Len, Requirements_Len, Benefits_Len, Company_Profile_Word_Count, Description_Word_Count, Requirements_Word_Count, Benefits_Word_Count	9

Figure 17: Tables of chosen columns to use for training

2.4 Prepping Dataset

We use **BeautifulSoup** and **Re** to strip our free text columns, **removing line terminators, urls, emails, phone numbers, non-english words, bullets and contractions**. We also

split merged words (often in technical jargon) to reduce the amount of unique words in our dataset, helping our embeddings to learn more meaningful correlations.

We then remove outliers in our dataset, those below 25% quantile and above 75% quantile of the non-binary numeric columns. The finalized dataset is saved in **data/clean/fake_job_postings.ALL.csv**

3. Embeddings

3.1 Data Preprocessing

We split our dataset into **80:10:10 Train:Test:Validation** allowing our model to learn custom embeddings from our dataset. We then do word tokenization using **nlTK.tokenize** by

```
sentences = [word_tokenize(text.lower()) for text in X_train_text]
```

Figure 18: Code to tokenize text samples from train dataset

3.2 FastText vs CBoW

We load both models from **Gensim.models** with the same initial parameters, building the corpus from our train set.

```
fasttext_model = FastText(vector_size=100, window=5, min_count=5, sg=1)
fasttext_model.build_vocab(sentences)
fasttext_model.train(sentences, total_examples=fasttext_model.corpus_count,
epochs=10)
```

```
cbow_model = Word2Vec(vector_size=100, window=5, min_count=5, sg=0)
cbow_model.build_vocab(sentences)
cbow_model.train(sentences, total_examples=cbow_model.corpus_count,
epochs=10)
```

Figure 19: Code to fine tune FastText and CBoW models

We benchmark both models against **10 top most similar words** and **cosine similarity between words**.

```
print(model.wv.similarity('software', 'engineer'))
print(model.wv.similarity('skills', 'experience'))
model.wv.most_similar('research', topn=10)
```

FastText	CBoW
----------	------

0.45782378 0.52482945 [('researches', 0.690284013748169), ('researched', 0.6851828694343567), ('qualitative', 0.6454797983169556), ('researching', 0.6369368433952332), ('quantitative', 0.6147993803024292), ('researcher', 0.6009271144866943), ('studies', 0.594095766544342), ('researchers', 0.5761727094650269), ('statistical', 0.5751654505729675), ('qualitatively', 0.5732088685035706)]	0.07083757 0.36082172 [('qualitative', 0.5862869620323181), ('competitor', 0.5728808641433716), ('researching', 0.5130829811096191), ('trends', 0.5081406235694885), ('analysis', 0.5014487504959106), ('benchmarking', 0.49394726753234863), ('analyses', 0.47809627652168274), ('quantitative', 0.47441336512565613), ('segmentation', 0.47383177280426025), ('experiments', 0.46182969212532043)]
---	---

Figure 20: Tables of chosen columns to use for training

We can see that FastText has the higher correlation between top 10 words, so we will be using it to train our embeddings.

2.2.2. Hyperparameter Tuning

We aim to tune the **sliding window size, embedding vector size, epochs, and negative sampling** to obtain the optimal embedding model which would be easily plugged into our model. We would measure through extrinsic and intrinsic methods, checking the mean of top 10 similar words and checking if 2 correlated and uncorrelated words are similar. We run the hyperparameter tuning cell where the results are stored in **/data/clean/embedding_tuning.csv**.

We then save our optimal model (**FastText(vector_size=100, window=3, min_count=2, epochs=10, negative=5, sg = 1)**) into **/optimal_fasttext.bin**

```
param_grid = {
    'vector_size': [100, 200],
    'window':      [3, 5, 10],
    'min_count':   [2, 5],
    'epochs':      [10, 20],
    'negative':    [5, 10],
}

test_words = ['engineer', 'manager', 'python', 'healthcare',
              'experience', 'salary']
job_analogies = [
    ('engineer', 'python', 'manager', 'java'),
    ('senior', 'engineer', 'junior', 'developer'),
    ('full_time', 'salary', 'part_time', 'hourly'),
    ('healthcare', 'nurse', 'finance', 'analyst'),
]
vocab = [word for sentence in sentences for word in sentence]
```

```

results = []

keys = list(param_grid.keys())
combos = list(product(*param_grid.values()))
print(f"Total combinations: {len(combos)} runs")

for combo in combos:
    params = dict(zip(keys, combo))
    model = FastText(**params, sg=1, min_n=3, max_n=6)

    model.build_vocab(sentences)
    model.train(sentences, total_examples=model.corpus_count,
epochs=params['epochs'])

    coherence = nearest_neighbour(model, test_words)
    oov        = oov_rate(model, vocab)
    analogy    = analogy_score(model, job_analogies)

    results.append({
        **params,
        'coherence': round(coherence, 4),
        'oov_rate':   round(oov, 4),
        'analogy':   round(analogy, 4),
    })
    print(f" {params} → coherence={coherence:.4f}, oov={oov:.4f},
analogy={analogy:.4f}")

results_df = pd.DataFrame(results)
results_df.to_csv('data/clean/embedding_tuning.csv', index=False)

```

Figure 21: Code to conduct hyperparameter tuning for FastText model

Optimal hyperparameters found

```

{'vector_size': 100, 'window': 3, 'min_count': 2, 'epochs': 10,
'negative': 5} → coherence=0.7873, oov=0.0000, analogy=0.0000

```

Figure 22: Hyperparameter tuning results for FastText model

4. Model Architecture

4.1 Model Blocks

4.1.1. BiGRU

A bidirectional BiGRU block was used as the main sequential encoder for the text branch. It processes the sequence of tokens in both forward and backward directions, so that each token representation is informed by the words that come before and after it, providing surrounding context in the vector representation rather than individual words alone. We chose a BiGRU because it is well suited for sequential text data while being more computationally efficient than more complex architectures like transformers, which may be unnecessary for a small dataset of our size. The bidirectional design is important because phrases in fraudulent postings can be ambiguous unless the full local context is considered from both directions. So we managed to capture contextual dependencies while remaining relatively lightweight and still effective on small to moderate sized datasets. However, one limitation would be that BiGRU struggles with long range dependencies across text, especially on long sequences. In our case, this limitation was less severe because the input text consisted mostly of short job-posting extracts and the later multi-head attention pooling layer further helped the model focus on the most informative parts of the sequence.

```
class BiGRUBlock(nn.Module):
    # 2 Layer Bidirectional GRU.

    def __init__(self, embed_dim, hidden_dim, dropout=0.3):
        super().__init__()
        self.gru = nn.GRU(
            input_size=embed_dim,
            hidden_size=hidden_dim,
            num_layers=2, #stacked GRU Layers
            batch_first=True,
            bidirectional=True, #capture context from both directions
            dropout=dropout,
        )

    def forward(self, x):
        # x: (batch, seq_len, embed_dim) tensor shape of x, number of
        samples, number of tokens in each sample, and embedding dimension
        # output: (batch, seq_len, hidden_dim * 2) because hidden states
        from both directions are concatenated
        output, _ = self.gru(x) # discard final hidden state h_n
        return output
```

Figure 23: Code for the BiGRUBlock

4.1.2. Multi-head Attention Pooling

Multi-head attention pooling was used after the BiGRU to convert the sequence of token-level hidden states into one fixed-length text representation, to be concatenated later with the numerical branch. The multi-head attention pooling was implemented because fake job postings may contain several different suspicious signals in different parts of the text, e.g. vague job descriptions, unrealistic benefits, unusual contact details, etc.. In order to avoid assigning the same level of importance to all tokens, we implemented multiple heads to capture and put focus on different patterns at once instead of one single attention mechanism to summarize the whole posting. This was implemented in hopes that the model receives a richer and more flexible text representation in the compact representation vector. However in our current setup, we found that the extra parameters and noise that comes with multiple heads led to poorer model performance. With too many attention heads, the model struggled to converge on noisier data and the greater complexity had a higher tendency to overfit on the training data, underperforming during validation, especially since the dataset had very few fake job postings. In the end, we settled on two attention heads to capture suspicious signals simultaneously without compromising too much of stability.

```
class MultiHeadAttentionPooling(nn.Module):
    def __init__(self, hidden_dim, num_heads=2):
        super().__init__()
        self.heads = nn.ModuleList([
            nn.Linear(hidden_dim * 2, 1) for _ in range(num_heads)
        ])
        # project concatenated heads back to original dim
        self.projection = nn.Linear(hidden_dim * 2 * num_heads,
hidden_dim * 2)
        self.dropout = nn.Dropout(0.3)

    def forward(self, gru_output, attention_mask):
        # gru_output: (batch, seq_len, hidden_dim * 2)
        head_outputs = []
        for head in self.heads:
            scores = head(gru_output).squeeze(-1) # (batch,
seq_len)
            scores = scores.masked_fill(attention_mask == 0,
float('-inf'))
            weights = torch.softmax(scores, dim=1).unsqueeze(-1) #
(batch, seq_len, 1)
            pooled = (gru_output * weights).sum(dim=1) #
(batch, hidden_dim * 2)
            head_outputs.append(pooled)

        combined = torch.cat(head_outputs, dim=1) # (batch, hidden_dim
* 2 * num_heads)
        #return self.projection(combined) # (batch,
```

```
hidden_dim * 2)
    return self.dropout(self.projection(combined))
```

Figure 24: Code for the Multihead Attention Pooling Block

4.1.3. Numerical Block

The numerical block was used to process the structured, non-text features of each job posting in parallel with the text branch. In our model, this block consists of two fully connected linear layers, each followed by a ReLU activation and dropout, allowing the network to transform the raw numerical inputs into a denser hidden representation before merging them with the text representation. We chose this design because the numerical features, while simpler than the textual input, may still contain useful signals such as posting patterns or metadata such as length of text or number of missing fields.

```
class NumericalBlock(nn.Module):
    #2 Linear Layers with ReLU for processing numerical features.

    def __init__(self, num_features, hidden_dim, dropout=0.3):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(num_features, hidden_dim),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(dropout),
        )

    def forward(self, x):
        # x: (batch, num_features)
        return self.layers(x) # (batch, hidden_dim)
```

Figure 25: Code for the Numerical Block

4.2 Model Implementation

The model implementation is defined via the FakeJobDetector class where the textual and numerical information are processed through a two branch architecture. We separate the processing of unstructured text from structured numerical features before both representation vectors are merged through a final classifier layer. The text branch is responsible for learning contextual patterns from tokenized job posting content, while the numerical branch extracts useful signals from engineered features. The two outputs are concatenated and processed through the classifier layer that produces a single logit which is then converted into a probability via sigmoid function.

4.2.1. Block Instantiation

During initialization, the model instantiates all major components required for the two-branch architecture. The text branch begins with an embedding layer that maps token IDs into dense vector representations, with the option to load pretrained embeddings if available. These embeddings are then passed into the two-layer BiGRU block instantiated with a dropout layer to force the network to avoid co-adaption and overfitting. Then the multi-head attention pooling block is instantiated to convert the sequence of BiGRU hidden states into a single fixed-length text representation. In parallel, the numerical block is instantiated with a dropout layer as well. Once both branches are defined, the model calculates the merged feature size and instantiates a final linear classifier that maps the concatenated text and numerical representations to a single output logit. The model is then moved to the specified device so training and inference can be performed.

```
class FakeJobDetector(nn.Module):

    # Two-branch architecture:
    # NLP branch: Embedding -> BiGRU -> Attention Pooling
    # Numerical branch: 2 Linear Layers with ReLU
    # Merge: Concatenate -> Linear -> output Logit

    def __init__(
        self,
        vocab_size,
        embed_dim,
        gru_hidden_dim,
        num_numerical_features,
        num_hidden_dim,
        dropout=0.3,
        pretrained_embeddings=None,
        device='cpu'
    ):
        """Initialize the model with given hyperparameters and
        architecture components."""
        super().__init__()
        self.device = torch.device(device)

        # NLP branch
        self.embedding = nn.Embedding(vocab_size, embed_dim,
padding_idx=1)
        if pretrained_embeddings is not None: #if provided, copy to
embedding layer, otherwise random init
            self.embedding.weight.data.copy_(pretrained_embeddings)

        self.bigru = BiGRUBlock(embed_dim, gru_hidden_dim, dropout)
        #self.attention = AttentionPooling(gru_hidden_dim)
        self.attention = MultiHeadAttentionPooling(gru_hidden_dim)
```

```

        # Numerical branch
        self.numerical = NumericalBlock(num_numerical_features,
num_hidden_dim, dropout)

        #Merge branch
        merge_input_dim = (gru_hidden_dim * 2) + num_hidden_dim
        self.classifier = nn.Linear(merge_input_dim, 1)

        self.to(self.device)

```

Figure 26: Code for the FakeJobDetector Class

4.2.2. Forward method

The forward method defines how data flows through the model during training and inference. The token IDs, attention mask, and numerical features are moved to the selected device.

In the text branch, the input token IDs are passed through the embedding layer to obtain dense token vectors, which are then processed by the BiGRU block to generate contextualized hidden states for each position in the sequence. These hidden states are then summarized by the multi-head attention pooling layer, which uses the attention mask to ignore padding tokens and produces a single fixed-length vector representing the textual content of the job posting.

At the same time, the numerical features are passed through the numerical block to produce a hidden numerical representation. The outputs of the two branches are then concatenated along the feature dimension and passed through the final classifier, which returns a single logit for each sample. This logit is not passed through a sigmoid activation inside the forward method, but later in the process by the sigmoid focal loss function during training.

```

def forward(self, input_ids, attention_mask, numerical_features):
    """Forward pass through the model. Returns raw logits (no
sigmoid) for use with criterion function later."""
    # Prep inputs to device
    input_ids = input_ids.to(self.device)
    attention_mask = attention_mask.to(self.device)
    numerical_features = numerical_features.to(self.device)

    # NLP branch
    embedded = self.embedding(input_ids) # (batch,
seq_len, embed_dim)
    gru_out = self.bigru(embedded) # (batch,
seq_len, gru_hidden * 2)
    nlp_out = self.attention(gru_out, attention_mask) # (batch,
gru_hidden * 2)

```

```

        # Numerical branch
        num_out = self.numerical(numerical_features)      # (batch,
num_hidden)

        # Merge
        merged = torch.cat([nlp_out, num_out], dim=1)
        logit = self.classifier(merged).squeeze(-1)      # (batch, 1)
-> (batch,)
        return logit #apply sigmoid outside the forward pass so doesn't
interfere with training

```

Figure 27: Code for the forward method

4.2.3 Fit method

The fit method implements the full training loop for the model, including optimization, validation, checkpointing and metric tracking across epochs. We have chosen the Adam optimizer with weight decay for regularization, to utilize momentum and adaptive learning rates to provide fast, stable convergence. The training loss, validation loss, training accuracy and validation accuracy are stored in record arrays as per standard procedure.

Each epoch sets the model to training mode, passes the batch through the forward method to compute raw logits, which are then converted to probabilities via the sigmoid focal loss with hyperparameters set to better handle the strong class imbalance and prioritize the minority fake class. After loss is computed, backpropagation is performed with gradient clipping to improve training stability and the optimizer updates the model parameters as per standard procedure again.

Once all training batches for the epoch are processed, the method switches the model to evaluation mode and computes validation loss and accuracy on the validation set without gradient updates. The best-performing model, via the lowest validation loss, is saved during training and restored at the end, ensuring that the final model corresponds to the strongest validation checkpoint rather than simply the final epoch.

```

def fit(self, dataloader, val_dataloader, num_epochs, learning_rate,
save_path="best_model.pt"):
    """Train the model using the provided dataloader and validation
dataloader."""

    optimizer = torch.optim.Adam(self.parameters()),
lr=learning_rate, weight_decay=1e-3)

    train_losses      = []
    val_losses        = []
    train_accuracy     = []
    val_accuracy       = []

```

```

best_val_loss    = float('inf')

for epoch in range(num_epochs):
    # --- Training ---
    self.train()
    total_train_loss = 0.0
    train_correct = 0
    train_total = 0

    for inputs, targets in dataloader:
        optimizer.zero_grad()
        input_ids      = inputs['input_ids']
        attention_mask  = inputs['attention_mask']
        numerical_features = inputs['numerical_features']
        targets         = targets.to(self.device).float()

        outputs = self.forward(input_ids, attention_mask,
numerical_features)
        loss = sigmoid_focal_loss(outputs, targets, alpha=0.75,
gamma=1.0, reduction='mean')
        loss.backward()
        torch.nn.utils.clip_grad_norm_(self.parameters(),
max_norm=1.0)
        optimizer.step()
        total_train_loss += loss.item()

        preds = (torch.sigmoid(outputs) >= 0.3).long()
        train_correct += (preds == targets.long()).sum().item()
        train_total   += targets.size(0)

    avg_train_loss = total_train_loss / len(dataloader)
    train_losses.append(avg_train_loss)
    train_accuracy.append(train_correct / train_total)

    # --- Validation ---
    self.eval()
    total_val_loss = 0.0
    val_correct = 0
    val_total = 0

    with torch.no_grad():
        for inputs, targets in val_dataloader:
            input_ids      = inputs['input_ids']
            attention_mask  = inputs['attention_mask']
            numerical_features = inputs['numerical_features']
            targets         = targets.to(self.device).float()

```

```

        outputs = self.forward(input_ids, attention_mask,
numerical_features)
        loss = sigmoid_focal_loss(outputs, targets,
alpha=0.75, gamma=1.0, reduction='mean')
        total_val_loss += loss.item()

        preds = (torch.sigmoid(outputs) >= 0.3).long()
        val_correct += (preds ==
targets.long()).sum().item()
        val_total += targets.size(0)

    avg_val_loss = total_val_loss / len(val_dataloader)
    val_losses.append(avg_val_loss)
    val_accuracy.append(val_correct / val_total)

    print(f"Epoch {epoch+1}/{num_epochs} | Train Loss:
{avg_train_loss:.4f} | Val Loss: {avg_val_loss:.4f}")

    # --- Save Best Model ---
    if avg_val_loss < best_val_loss:
        best_val_loss = avg_val_loss
        self.save(save_path)
        print(f" 🟢 Best model saved
(val_loss={best_val_loss:.4f})")
    else:
        print(f" 🟡 No improvement
(best_val_loss={best_val_loss:.4f})")

    self.load(save_path)
    print(f"\nRestored best model with
val_loss={best_val_loss:.4f}")
    return train_losses, val_losses, train_accuracy, val_accuracy

```

Figure 28: Code for the fit method

4.2.4 Evaluation

We have implemented two evaluation methods under our FakeJobDetector class, the main evaluate method to evaluate model performance on an unseen test set and the other being an evaluate_branches method to quantify the contribution the text branch and the numerical branch provide to the final classification result.

The evaluate() method runs the model in evaluation mode on a given dataloader, applies the sigmoid function to convert logits into probabilities and then converts these probabilities into binary predictions using a configurable decision threshold. This threshold is especially important in our anomaly classification task since it allows the balance between precision

and recall for the fake-job class to be adjusted more carefully. The predicted labels are then compared against the true labels using a classification report, which summarizes precision, recall, and F1-score for both real and fake postings.

```
def evaluate(self, dataloader, threshold=0.3):
    """Evaluate model performance on the given dataloader and print
    classification report."""

    self.eval()
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for inputs, targets in dataloader:
            input_ids = inputs['input_ids']
            attention_mask = inputs['attention_mask']
            numerical_features = inputs['numerical_features']
            targets = targets.to(self.device).float()

            outputs = self.forward(input_ids, attention_mask,
numerical_features)
            preds = (torch.sigmoid(outputs) >= threshold).long()
#  tunable threshold

            all_preds.extend(preds.cpu().tolist())
            all_labels.extend(targets.long().cpu().tolist())

    print(classification_report(all_labels, all_preds,
target_names=['Real', 'Fake']))
```

Figure 29: Code for the evaluate method

The `evaluate_branches()` method performs a simple ablation analysis by selectively zeroing out either the NLP branch, the numerical branch, or both before classification. This allows the model to be tested under different branch configurations so that the relative contribution of textual and numerical features can be examined more explicitly.

```
def evaluate_branches(self, dataloader, threshold=0.3):
    """Evaluate model performance with different branch combinations
    to understand contribution of each modality."""
    from sklearn.metrics import classification_report

    configs = {
```

```

        "Full model":      dict(zero_nlp=False, zero_num=False),
        "NLP only":       dict(zero_nlp=False, zero_num=True),
        "Numerical only": dict(zero_nlp=True,  zero_num=False),
        "Both zeroed":    dict(zero_nlp=True,  zero_num=True),
    }

    for name, cfg in configs.items():
        self.eval()
        all_preds, all_labels = [], []

        with torch.no_grad():
            for inputs, targets in dataloader:
                input_ids      = inputs['input_ids']
                attention_mask  = inputs['attention_mask']
                numerical_features =
inputs['numerical_features'].to(self.device)
                targets        = targets.to(self.device).float()

                # Forward pass with branch control
                embedded = self.embedding(input_ids.to(self.device))
                gru_out  = self.bigru(embedded)
                nlp_out  = self.attention(gru_out,
attention_mask.to(self.device))
                num_out  = self.numerical(numerical_features)
                # Zero out branches as per config
                if cfg["zero_nlp"]:
                    nlp_out = torch.zeros_like(nlp_out)
                if cfg["zero_num"]:
                    num_out = torch.zeros_like(num_out)

                merged = torch.cat([nlp_out, num_out], dim=1)
                logit  = self.classifier(merged).squeeze(-1)
                preds  = (torch.sigmoid(logit) >= threshold).long()

                all_preds.extend(preds.cpu().tolist())
                all_labels.extend(targets.long().cpu().tolist())

        print(f"\n{'='*40}\n{name}")
        print(classification_report(all_labels, all_preds,
target_names=['Real', 'Fake']))

```

Figure 30: Code for the `evaluate_branches` method

5. Model Training

5.1 Data Preprocessing

Our model architecture expects the inputs to be a series of `input_ids`, `attention_masks` and `numerical_features`. These are numerical representations that we need to convert our cleaned dataset into before we can start to train our model.

5.1.1. Train Test Validation Split

A 80 - 10 - 10 split is used to create the train, validation and test datasets. They are run with seed 42 for reproducibility and are stratified with respect to the label class distribution. This is to achieve the same class distribution within each dataset so as to prevent bias evaluation and ensure reliability of performance metrics during evaluation.

5.1.2. Non-binary Value Standardisation

For each non-binary numerical column in the train dataset, they are standardised and fitted using the **StandardScaler** function from **sklearn**. This is to reduce feature dominance by their magnitudes and improve the convergence rate and performance during model training.

The same StandardScaler is then used to transform the validation and test dataset. The StandardScaler is not fitted to the validation and test to prevent data leakage and to simulate the model experiencing new and unseen data based on the distribution and parameters learnt from the training data itself.

5.1.3. Tokenization, Encoding, Embedding Matrix

After loading the fine-tuned embedding model from FastText, the samples in each dataset are tokenized into n-grams characters. They are then encoded with the vocabulary learnt from the embedding model. These numerical representations and vocab are then cached as a pickle file for efficiency.

Additionally, the pretrained embedding matrix for the model architecture is created by instantiating a zero value matrix using the vocabulary size and the vector size of the embedding model. Subsequently, it uses the vocabulary indexes and numerical values and replaces the 0s in the embedding matrix with the numerical values. The padding tokens will get a numerical value of 0 in the matrix to ignore the paddings used in the dataset class.

This creates an embedding matrix that will be taken up by the model to generate an embedding vector upon receiving numerical sequences as inputs.

5.1.3. Dataset Dataloader Instantiation

After preprocessing the necessary data inputs, they are then used to instantiate the respective train, validation and test datasets. The dataset class **JobPostingDataset** is customised to contain the numerical representations of tokenized text, the numerical features and the labels. Upon iterating the batches, the dataset class will trim the tokens to a

predefined maximum length. Since the architecture expects a fixed length, if a sample's entire length is less than the max length, it will be padded with padding tokens. Lastly, the final numeric input values will each be stored in tensors which are used by the model.

```
import torch
from torch.utils.data import Dataset, DataLoader

class JobPostingDataset(Dataset):
    def __init__(self, texts_tok, numerical_features, labels, max_len=2048):
        self.texts_tok = texts_tok  # already tokenized integer
        sequences
        self.numerical_features = numerical_features  # (N, num_features)
        array or dataframe
        self.labels = labels
        self.max_len = max_len

    def __len__(self):
        return len(self.texts_tok)

    def __getitem__(self, idx):
        tokens = self.texts_tok[idx][:self.max_len]
        label = self.labels[idx]

        # Pad to max_len
        pad_len = self.max_len - len(tokens)
        attention_mask = [1] * len(tokens) + [0] * pad_len
        tokens = tokens + [1] * pad_len  # 1 = <pad> index

        inputs = {
            'input_ids': torch.tensor(tokens,
dtype=torch.long),
            'attention_mask': torch.tensor(attention_mask,
dtype=torch.long),
            'numerical_features':
torch.tensor(self.numerical_features[idx], dtype=torch.float),
        }
        label = torch.tensor(label, dtype=torch.float)  # float for BCELoss
        return inputs, label
```

Figure 31: Code for the JobPostingDataset Class

The respective dataloader class will take in the respective datasets and use a predefined batch_size number. Only the train_dataloader will be shuffled so as to prevent the model from memorizing the dataset.

5.2 Model Training

After instantiating both the datasets and their dataloaders, the model is ready to be called and trained.

5.2.1. Model Instantiation

To instantiate the model, we will first pass a set of parameters.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = FakeJobDetector(
    vocab_size=vocab_size,
    embed_dim=embed_dim,
    num_numerical_features=X_train_numeric.shape[1],
    pretrained_embeddings=pretrained_embeddings,
    gru_hidden_dim=64,
    num_hidden_dim=128,
    device=device,
)

print(model)
```

Figure 32: Code to instantiate the model class

- **vocab_size** is the length of the vocabulary built from FastText.
- **embed_dim** is the vector size of the fine-tuned FastText model.
- **num_numerical_features** is the number of numerical columns in the X_train_numeric data.
- **pretrained_embeddings** is the embedding matrix we have created previously.
- **gru_hidden_dim** is a chosen dimension to use for the BiGRU block.
- **num_hidden_dim** is a chosen dimension to use for the numerical block.
- **device** is to ensure that all operations happening in the model are of the same device.

5.2.2. Training Loop

The model calls the fit() method to start the training loop.

```
train_losses, val_losses, train_accuracy, val_accuracy = model.fit(
    dataloader      = train_dataloader,
    val_dataloader  = val_dataloader,
    num_epochs      = 20,
    learning_rate   = 1e-3,
    save_path       = "best_model.pt",
)
```

Figure 33: Code to train the model

- The code above is how to use the training method on the model, to train it from scratch
- Ensure that the necessary train and validation dataloaders are there
- Select a **num_epochs** and **learning_rate** value
- Provide the **save_path** of the weights.

5.2.3. Test Set Evaluation

After training, the model is loaded with the saved weights that contain the lowest validation loss. The model is then tested against the test dataset with a threshold of 0.5.

```
model.evaluate(test_dataloader, threshold=0.5)
```

```
--- Threshold: 0.5 ---
              precision    recall  f1-score   support

   Real         0.99         1.00         1.00        1702
   Fake         0.95         0.86         0.90         86

 accuracy              0.99              1788
 macro avg         0.97         0.93         0.95        1788
 weighted avg      0.99         0.99         0.99        1788
```

Figure 34: Code to evaluate the model with the test dataset and its results

5.2.4. Result Visualizations

Multiple visualisations have been created from the stored training and loss validation values so as to visualise the training progress of the model. Additionally, confusion matrices are generated to visualise the test evaluation results so as to understand how the model is performing under unseen data.

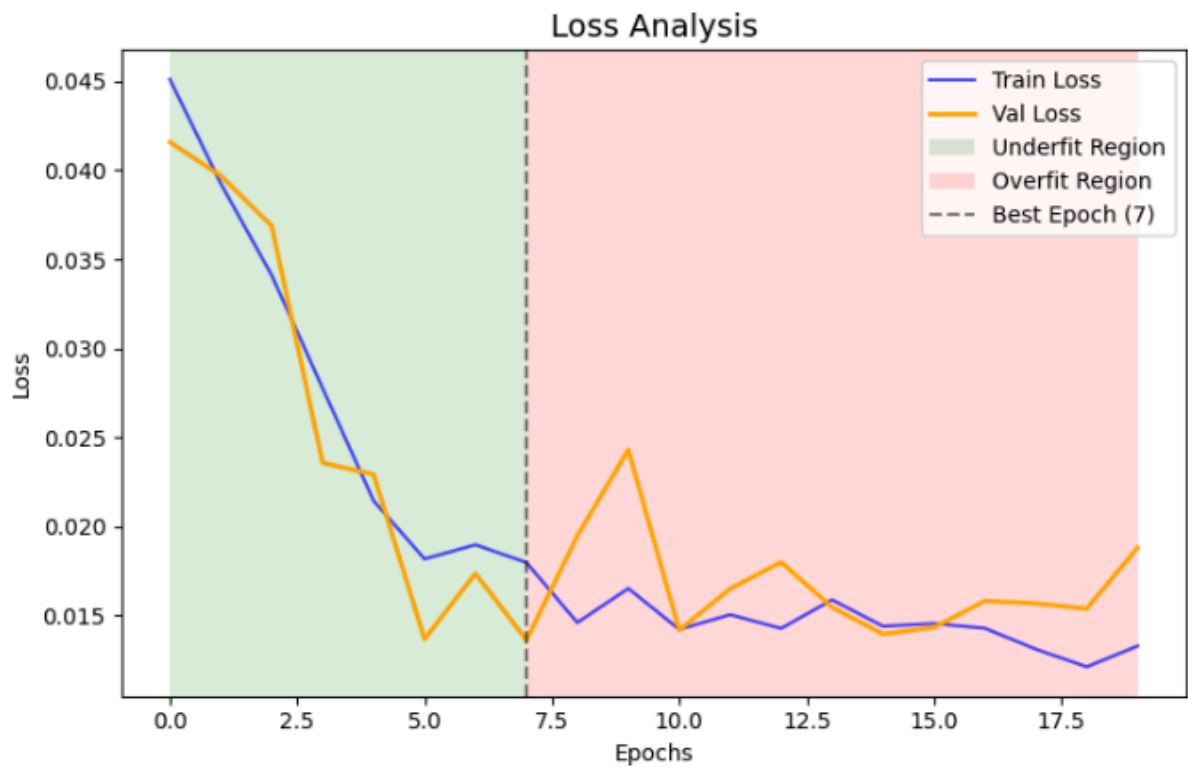


Figure 35: Loss Analysis Curve

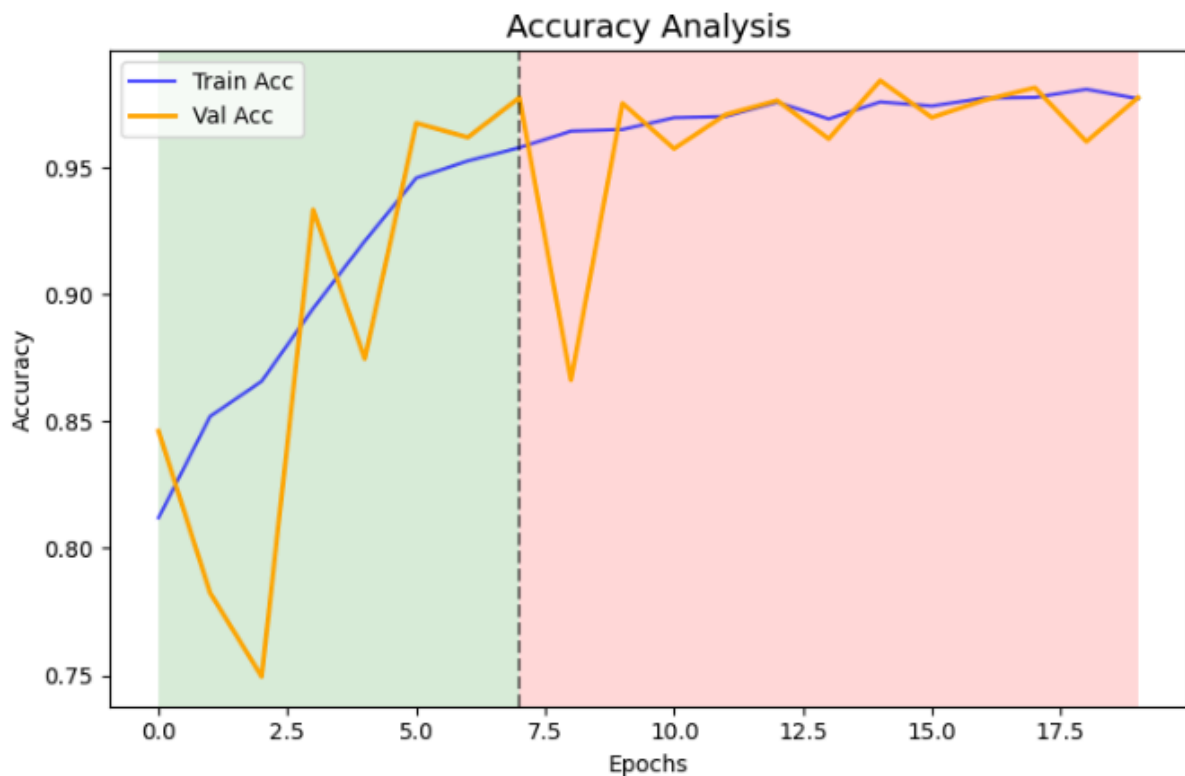


Figure 36: Accuracy Analysis Curve

- Loss Analysis shows that the model is indeed training with a continuous decrease in train loss value
- The validation loss tends to decrease as well up to a certain point. The model started to overfit after epoch 7 where the validation loss and the training loss started to diverge in values.
- The areas of when the model is still underfitting and has yet to fully train and when the model is overfitting are indicated with the green and red zones respectively.
- Results such as this shows why a saver and loader function is necessary so as to load the model with the weights that gives the best generalization ability overall.

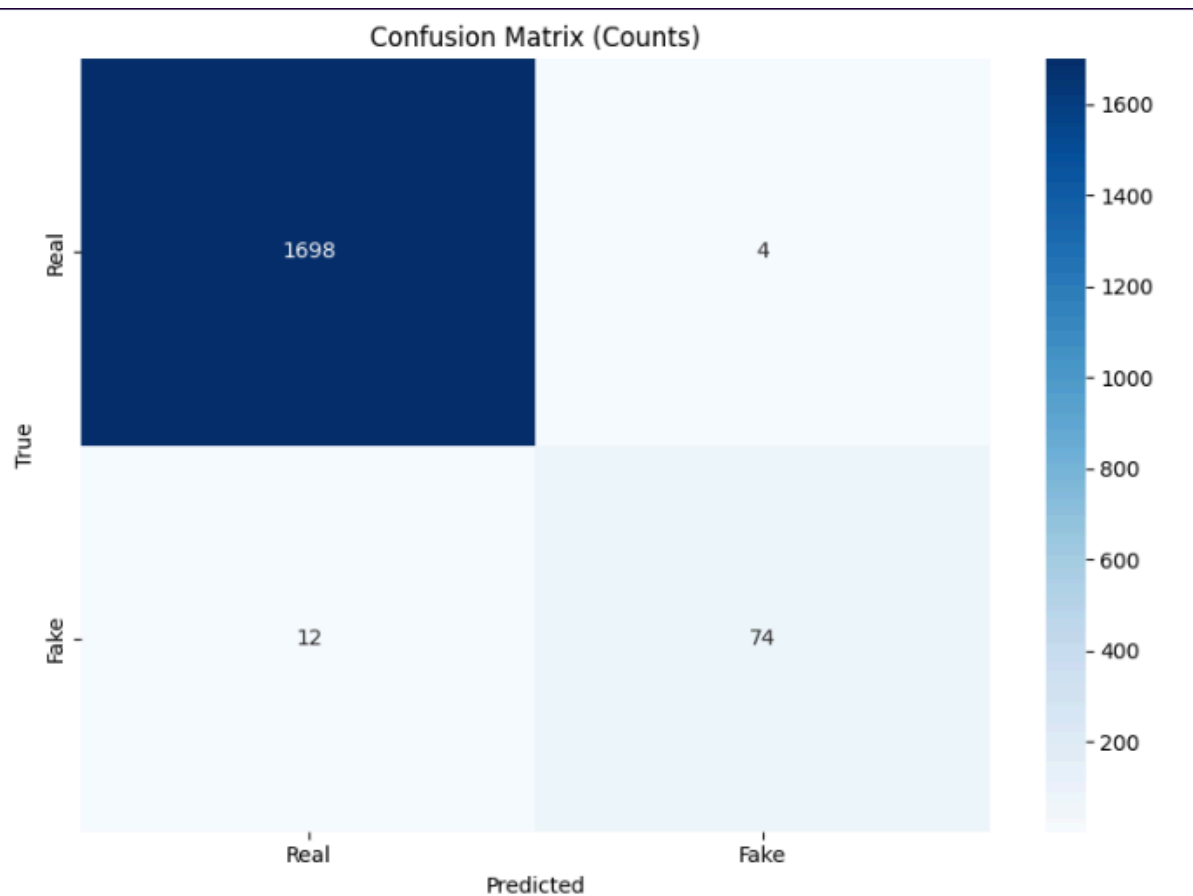


Figure 37: Confusion Matrix with raw counts

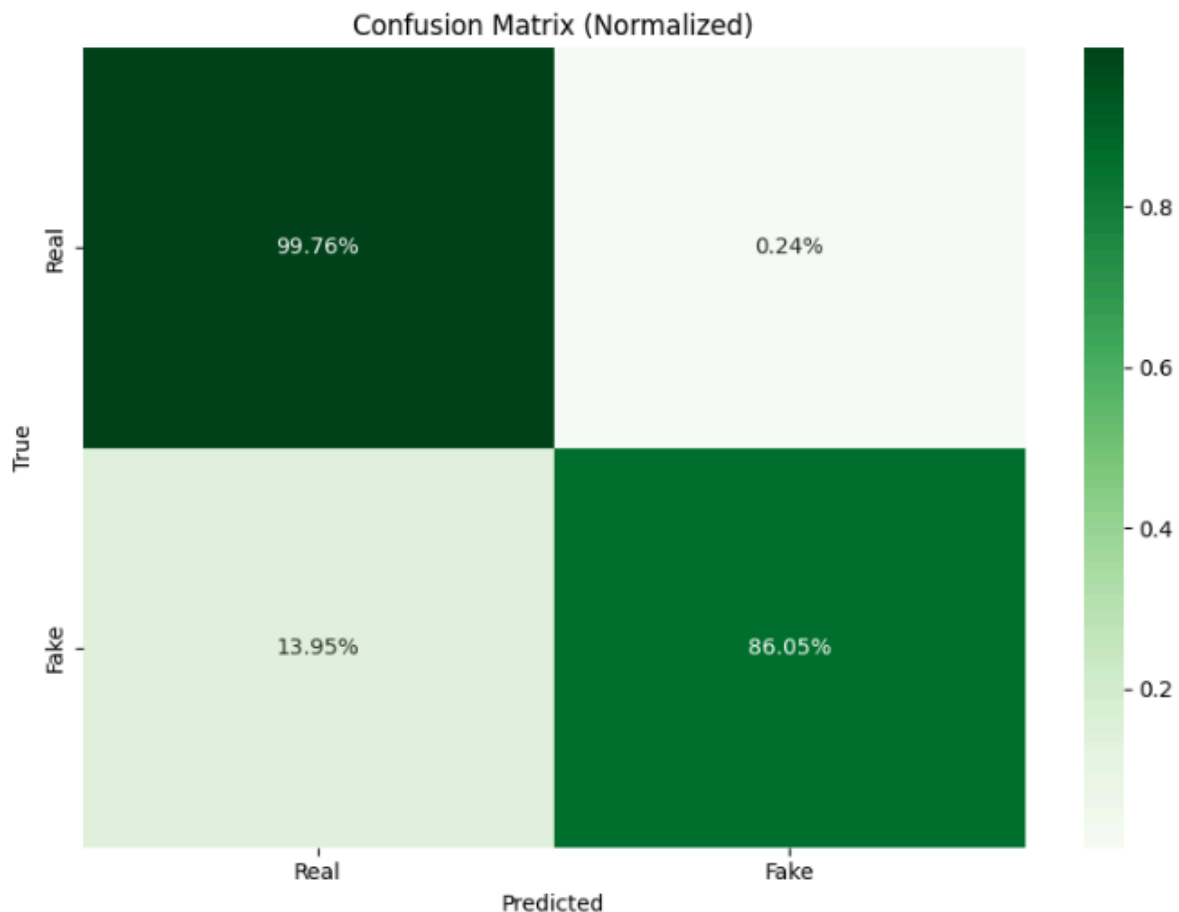


Figure 38: Confusion Matrix with normalized values

- The confusion matrices showed how the model performed with the test dataset.
- The matrices showed both the raw counts and percentages of the dataset that the model has correctly or incorrectly classified the test samples
- For instance, the model correctly classified real posts 99.76% of the time and fake posts 86.05% of the time.
- This means that the model may misclassify a fake post to be real 13.95% of the time and may misclassify a real post to be fake 0.24% of the time.
- Note that this is implicitly done with a sigmoid threshold of 0.5.

To obtain the figures that you have seen above, ensure that “**fake_job_postings_ALL.csv**” is in the “**/data/clean**” directory. Under **Model_training.ipynb**, run all of the following cells from “**Dataset Loading**” to “**Model Training**”. After the model has been trained and has loaded the best weights, run the cells under “**Loss and Accuracy Visualisations**” and “**Confusion Matrix Report Visualisation**”. They will return the loss and accuracy curves and the confusion matrix generated in the report.

Avoid running the “**Hyperparameter GRU/Hidden layer dimension tuning**”, this will take quite some time to run and our best track record was 5 hours.

5.2.5. Hyperparameter Tuning

To obtain the best model results, a series of hyperparameter tuning had to be done.

Firstly, we will want to adjust our sigmoid threshold value so as to obtain the best recall value without impacting the F1 score greatly. The F1 score is used as a generic metric to evaluate our model's ability to maintain a high precision and recall score. Hence, obtaining a sufficiently high F1 score is the first indication of a well-performing model.

However, in our project's context, classifying a fake post to be real has serious consequences, leading to a need to minimize the occurrence of false negatives. Therefore there is a need to adjust the threshold to balance the tradeoff between precision and recall, where we can obtain the best recall value without severely impacting the F1 score. This is the second and the most important indication of a well-performing model for this project.

```
--- Threshold: 0.2 ---
      precision    recall  f1-score   support

    Real         0.99      0.98      0.99      1702
    Fake         0.66      0.90      0.76        86

 accuracy              0.97      1788
 macro avg         0.83      0.94      0.87      1788
 weighted avg      0.98      0.97      0.97      1788

--- Threshold: 0.3 ---
      precision    recall  f1-score   support

    Real         0.99      1.00      0.99      1702
    Fake         0.90      0.87      0.89        86

 accuracy              0.99      1788
 macro avg         0.95      0.93      0.94      1788
 weighted avg      0.99      0.99      0.99      1788

--- Threshold: 0.4 ---
      precision    recall  f1-score   support

    Real         0.99      1.00      1.00      1702
    Fake         0.95      0.86      0.90        86

 accuracy              0.99      1788
 macro avg         0.97      0.93      0.95      1788
 weighted avg      0.99      0.99      0.99      1788
```

```

--- Threshold: 0.5 ---
      precision    recall  f1-score   support

    Real         0.99      1.00      1.00      1702
    Fake         0.95      0.86      0.90        86

 accuracy         0.99         0.99      1788
macro avg         0.97      0.93      0.95      1788
weighted avg         0.99      0.99      0.99      1788

```

```

--- Threshold: 0.55 ---
      precision    recall  f1-score   support

    Real         0.99      1.00      1.00      1702
    Fake         0.95      0.85      0.90        86

 accuracy         0.99         0.99      1788
macro avg         0.97      0.92      0.95      1788
weighted avg         0.99      0.99      0.99      1788

```

```

--- Threshold: 0.6 ---
      precision    recall  f1-score   support

    Real         0.99      1.00      0.99      1702
    Fake         0.95      0.83      0.88        86

 accuracy         0.99         0.99      1788
macro avg         0.97      0.91      0.94      1788
weighted avg         0.99      0.99      0.99      1788

```

```

--- Threshold: 0.7 ---
      precision    recall  f1-score   support

    Real         0.99      1.00      0.99      1702
    Fake         0.96      0.80      0.87        86

 accuracy         0.99         0.99      1788
macro avg         0.97      0.90      0.93      1788
weighted avg         0.99      0.99      0.99      1788

```

```

--- Threshold: 0.8 ---
      precision    recall  f1-score   support

```

Real	0.99	1.00	0.99	1702
Fake	0.97	0.74	0.84	86
accuracy			0.99	1788
macro avg	0.98	0.87	0.92	1788
weighted avg	0.99	0.99	0.99	1788

Figure 39: Hyperparameter Tuning with sigmoid thresholds

Secondly, the `gru_hidden_dim` and `num_hidden_dim` are specifically fine-tuned so that the model is able to best capture the variance and signals of the data. This is done by creating different combinations of parameter values among a set of values to experiment from.

Thereafter, we will train each model with these combinations for 20 epochs and a learning rate of $1e-3$. The best validation loss and weights of each model is recorded and stored in a list. This would return the model with the best validation loss and weights with its corresponding parameters.

```
# Define search space
gru_dims      = [32, 64]
num_dims      = [32, 64, 128]
combinations = list(itertools.product(gru_dims, num_dims)) # 6 combinations

results = []

for gru_hidden_dim, num_hidden_dim in combinations:
    print(f"\n{'='*50}")
    print(f"Training: gru_hidden_dim={gru_hidden_dim},
num_hidden_dim={num_hidden_dim}")
    print(f"{'='*50}")

    # Fresh model for each combination
    model = FakeJobDetector(
        vocab_size      = vocab_size,
        embed_dim       = embed_dim,
        gru_hidden_dim  = gru_hidden_dim,
        num_numerical_features = X_train_numeric.shape[1],
        num_hidden_dim  = num_hidden_dim,
        pretrained_embeddings = pretrained_embeddings,
        device          = device,
    ).to(device)

    save_path = f"model_gru{gru_hidden_dim}_num{num_hidden_dim}.pt"

    train_losses, val_losses = model.fit(
        dataloader      = train_dataloader,
```

```

        val_dataloader = val_dataloader,
        num_epochs      = 20,
        learning_rate   = 1e-3,
        save_path       = save_path,
    )

    best_val_loss = min(val_losses)

    # Evaluate on test set with a fixed threshold (e.g., 0.5) to compare
    across models
    print(f"\n--- Threshold: 0.5 ---")
    model.evaluate(test_dataloader, threshold=0.5)

    results.append({
        "gru_hidden_dim" : gru_hidden_dim,
        "num_hidden_dim" : num_hidden_dim,
        "best_val_loss"   : best_val_loss,
        "save_path"       : save_path,
    })

    print(f"Best val loss: {best_val_loss:.4f}")

# rank results by best_val_loss
results.sort(key=lambda x: x["best_val_loss"])

print("\n===== HYPERPARAMETER TUNING RESULTS =====")
for rank, r in enumerate(results, 1):
    print(f"# {rank} | gru={r['gru_hidden_dim']}, num={r['num_hidden_dim']} "
          f"| val_loss={r['best_val_loss']:.4f} | saved: {r['save_path']}")

# Best combo
best = results[0]
print(f"\nBest config → gru_hidden_dim={best['gru_hidden_dim']}, "
      f"num_hidden_dim={best['num_hidden_dim']} "
      f"(val_loss={best['best_val_loss']:.4f})")

```

Figure 40: Hyperparameter Tuning of Dimension Parameters in the model

```

=====
Training: gru_hidden_dim=64, num_hidden_dim=64
=====

Restored best model with val_loss=0.0125

--- Threshold: 0.5 ---
           precision    recall  f1-score   support

```

Real	1.00	0.99	0.99	1702
Fake	0.86	0.92	0.89	86
accuracy			0.99	1788
macro avg	0.93	0.96	0.94	1788
weighted avg	0.99	0.99	0.99	1788

Best val loss: 0.0125

===== HYPERPARAMETER TUNING RESULTS =====

```
#1 | gru=64, num=64 | val_loss=0.0125 | saved: model_gru64_num64.pt
#2 | gru=64, num=128 | val_loss=0.0136 | saved: model_gru64_num128.pt
#3 | gru=32, num=32 | val_loss=0.0146 | saved: model_gru32_num32.pt
#4 | gru=64, num=32 | val_loss=0.0154 | saved: model_gru64_num32.pt
#5 | gru=32, num=128 | val_loss=0.0395 | saved: model_gru32_num128.pt
#6 | gru=32, num=64 | val_loss=0.0401 | saved: model_gru32_num64.pt
```

Best config → gru_hidden_dim=64, num_hidden_dim=64 (val_loss=0.0125)

Figure 41: Hyperparameter Tuning Results of Dimension Parameters

5.2.6. Load Finalized Model

Saver and Loader functions are created to save and load trained model weights for both reproducibility and efficiency. The saver function is used to save the best current weights during training. This is so that once the model starts to overfit and perform badly, there is no need to retrain the model and we can use the saved weights to resume training or to evaluate.

This is done with the **torch.save(self.state_dict(),path)** which will save the parameters of the model to a “.pt” file

The loader function ensures reproducibility by loading the best trained weights so that we can obtain the best model without the need to retrain. This is done with the **self.load_state_dict(torch.load(path,map_location=self.device))** which will load the saved parameters into the model.

Henceforth, we use the saver function to save the best weights of the finalized model and use the loader function to load the weights into a newly instantiated model so that we can evaluate its performance with the test set.

```
def save(self,path):
    """ Implement model saving Logic here
    torch.save(self.state_dict(), path)
    return

def load(self,path):
```

```

    ### Implement model Loading Logic here
    self.load_state_dict(torch.load(path,map_location=self.device))
    self.to(self.device) #load to cpu by default, can be modified to
load to gpu if needed
    return

```

Figure 42: Code of Saver and Loader Functions

6. Model Evaluation

6.1 Model Experimentation

6.1.1. Distilbert Model

To investigate whether richer pretrained language representations could improve the fake job detection, we conducted an additional experiment involving DistilBERT's 768 dimension contextual embeddings over our 100 dimension fast text embeddings. The rationale behind this experiment was that DistilBERT provides contextualized embeddings, meaning that the representation of a word changes depending on the words around it. In principle, this should help the model better capture ambiguity, phrasing and contextual cues in fraudulent postings, especially when suspicious language depends on surrounding wording rather than isolated keywords. To keep the overall architecture compatible to an extent, the DistilBERT embeddings were used as the text representation layer and then passed into the existing BiGRU, multi-head attention pooling and numerical feature branches.

However, this variation performed worse than the Fasttext based model.

--- Threshold: 0.2 ---				
	precision	recall	f1-score	support
Real	0.99	0.96	0.97	1280
Fake	0.52	0.87	0.65	69
accuracy			0.95	1349
macro avg	0.75	0.91	0.81	1349
weighted avg	0.97	0.95	0.96	1349
--- Threshold: 0.3 ---				
	precision	recall	f1-score	support
Real	0.99	0.97	0.98	1280
Fake	0.57	0.84	0.68	69
accuracy			0.96	1349

macro avg	0.78	0.90	0.83	1349
weighted avg	0.97	0.96	0.96	1349
--- Threshold: 0.4 ---				
	precision	recall	f1-score	support
Real	0.99	0.97	0.98	1280
Fake	0.64	0.83	0.72	69
accuracy			0.97	1349
macro avg	0.82	0.90	0.85	1349
weighted avg	0.97	0.97	0.97	1349
--- Threshold: 0.5 ---				
	precision	recall	f1-score	support
Real	0.99	0.98	0.99	1280
Fake	0.71	0.81	0.76	69
accuracy			0.97	1349
macro avg	0.85	0.90	0.87	1349
weighted avg	0.98	0.97	0.97	1349

Figure 43: Evaluation Results from Distilbert model

Firstly, the DistilBERT encoder was used in a frozen setting rather than being fully fine-tuned, which limited its ability to adapt to the specific language patterns of our dataset.

Secondly, DistilBERT imposes a maximum sequence length of 512 tokens, which may have truncated useful information from longer job postings, whereas the original FastText-based pipeline could retain more of the posting text.

Thirdly, further processing the text with a BiGRU after it had already been encoded by DistilBERT likely introduced additional complexity without providing enough complementary benefit. Both DistilBERT and the BiGRU are designed to capture contextual information across the token sequence, so stacking them may have created redundancy in the text-processing pipeline, leading to poorer performance and the idea was not pursued further.

6.1.2. Numerical and NLP Branch Evaluation

The model method **evaluate_branches** is a function that evaluates model performance with respect to each branch to understand contributions to the overall performance.

This is done by passing the inputs to the model and before the results from the NLP and numerical branch merge, we will zero out the results of the other branch when evaluating a

branch. For instance, we will zero out the results of the numerical branch when evaluating the NLP branch.

=====				
Full model				
	precision	recall	f1-score	support
Real	0.99	1.00	1.00	1702
Fake	0.95	0.86	0.90	86
accuracy			0.99	1788
macro avg	0.97	0.93	0.95	1788
weighted avg	0.99	0.99	0.99	1788
=====				
NLP only				
	precision	recall	f1-score	support
Real	0.99	1.00	1.00	1702
Fake	0.95	0.85	0.90	86
accuracy			0.99	1788
macro avg	0.97	0.92	0.95	1788
weighted avg	0.99	0.99	0.99	1788
=====				
Numerical only				
	precision	recall	f1-score	support
Real	0.97	0.94	0.95	1702
Fake	0.26	0.41	0.32	86
accuracy			0.92	1788
macro avg	0.61	0.67	0.64	1788
weighted avg	0.94	0.92	0.92	1788

Figure 44: Evaluation Results from the `evaluate_branches` method

The outputs show that the branch that is performing natural language processing is contributing the most of the scores as compared to the numerical branch. Although the numerical branch contributes very little to the overall scores, it did assist in increasing the recall score slightly by 0.01.

6.1.3. Loss Function Experimentation

Sigmoid_focal_loss is used to address the extreme class imbalance between the real and fake classes. It reduces the loss generated from high confident and correct classifications but amplifies high confident misclassifications. Thus, we will need to adjust these hyperparameters to achieve the best results for our model.

```
sigmoid_focal_loss(outputs, targets, alpha=0.75, gamma=1.0,  
reduction='mean')
```

Figure 45: Code of sigmoid focal loss function

The gamma parameter controls the loss calculated. By increasing gamma, the loss generated from confident and correct classifications will be reduced exponentially and increased exponentially for confident and incorrect classifications.

The alpha parameter balances the importance of the classes, it instills a weighting factor of α to the positive class and a factor of $1-\alpha$ to the negative class. This is used so that misclassifying a fake post will lead to higher loss obtained.

Therefore, after trying multiple combinations of alpha and gamma, an alpha of 0.75 and a gamma of 1 lead to the best model performance.

6.2 Comparison against similar Kaggle Models

6.2.1. Bi-LSTM Architecture

We compared against:

<https://www.kaggle.com/code/gauravsahani/real-or-fake-job-postings-with-bi-directional-lstm#If-you-like-my-approach.-please-give-a-UpVote>

It was created with **one-hot encoding**, with a **single bi-lstm** and **binary cross entropy as the loss**:

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 40, 50)	250000
bidirectional (Bidirectional)	(None, 200)	120800
dropout (Dropout)	(None, 200)	0
dense (Dense)	(None, 1)	201
Total params: 371,001		
Trainable params: 371,001		
Non-trainable params: 0		

Figure 46: Model Architecture of Bi-LSTM solution

Its classification report is shown below:

	precision	recall	f1-score	support
0	0.97	0.99	0.98	3585
1	0.79	0.52	0.63	212
accuracy			0.97	3797
macro avg	0.88	0.76	0.81	3797
weighted avg	0.96	0.97	0.96	3797

Figure 47: Evaluation Results of Bi-LSTM solution

The most likely reasons for the difference includes:

- Using **sigmoid_focal_loss as our loss function** compared to vanilla cross entropy, where the former down-weights easy negatives (real jobs the model can classify correctly) and forces the model to focus on learning hard positives, crucial for an imbalanced dataset. Explains why there is a recall gap of 0.85 vs 0.52
- Using **BiGRU + Attention Pooling** vs simple BiLSTM: BiLSTM collapses the whole sequence into 1 vector via the final hidden state, losing a lot of mid-sequence signal, which is prevented by using MultiHeadAttention Pooling, learning which tokens matter most

6.2.2. BERT Architecture

We compared against:

<https://www.kaggle.com/code/ahmed0sultan/text-classification-using-bert-99-acc-98-7-f1>

It was **only trained on the free-text columns**, with **no fine-tuning** of the model, and **used generic pre-trained embeddings**.

```
from simpletransformers.classification import ClassificationModel
model = ClassificationModel('bert', 'bert-base-uncased', num_labels=2,
args={'overwrite_output_dir': True, "train_batch_size": 64, "save_steps":
10000, "save_model_every_epoch": False,
'num_train_epochs': 4}, use_cuda=True)
```

Figure 48: Model Architecture of BERT solution

Its classification report is shown below:

	precision	recall	f1-score	support
0	0.99	1.00	0.99	3403
1	0.89	0.80	0.84	173
accuracy			0.99	3576
macro avg	0.94	0.90	0.92	3576
weighted avg	0.99	0.99	0.99	3576

Figure 49: Evaluation Results of BERT solution

The most likely reasons for the difference includes:

- **No hyperparameter tuning involved:** In most scenarios, BERT would beat BiGRU due to it having 110M parameters of pre-trained knowledge, self-attention across all layers. However, its pre-trained embeddings might be lacking in technical jargon (compared to our custom embeddings on corpus), lacking in threshold, alpha/gamma and other hyperparameter tuning. Having a numerical branch could also increase its recall.

6.3 Malfunction

6.3.1. White Box Adversarial Attack

A white box adversarial attack was performed on the model to intentionally induce malfunction, stress test and identify its current limits. We investigated if the model was susceptible to small, targeted perturbations under full access to the model's architecture, parameters and gradients.

In our investigation, the attacker is assumed to know the internal structure of the classifier and can compute how small changes to the input representation affect the final real/fake prediction. The attack was applied to the embedding representation of the input text using the Fast Gradient Sign Method (FGSM), where the input embeddings were adjusted in the direction that would reduce the model's confidence in the correct fake-job classification.

```
# FGSM Attack
epsilon = 0.3

inputs, label = test_dataset[sample_idx]
input_ids = inputs['input_ids'].unsqueeze(0).to(device)
attention_mask = inputs['attention_mask'].unsqueeze(0).to(device)
numerical_features =
inputs['numerical_features'].unsqueeze(0).to(device)

# Must be in train mode for GRU backward, but weights are still frozen
model.train()

# Get embeddings and enable gradient on them
embedded = model.embedding(input_ids)
embedded = embedded.detach().requires_grad_(True)

# Forward pass bypassing embedding layer
gru_out = model.bigru(embedded)
nlp_out = model.attention(gru_out, attention_mask.to(device))
num_out = model.numerical(numerical_features)
merged = torch.cat([nlp_out, num_out], dim=1)
logit = model.classifier(merged).squeeze(-1)

# Compute gradient
```

```

logit.backward()

# FGSM perturbation
perturbed = embedded.detach() - epsilon * embedded.grad.sign()

# Forward with perturbed embeddings
model.eval()
with torch.no_grad():
    gru_out2 = model.bigru(perturbed)
    nlp_out2 = model.attention(gru_out2, attention_mask.to(device))
    num_out2 = model.numerical(numerical_features)
    merged2 = torch.cat([nlp_out2, num_out2], dim=1)
    logit2 = model.classifier(merged2).squeeze(-1)
    new_prob = torch.sigmoid(logit2).item()

```

Figure 50: Code to conduct the FGSM Attack

The perturbed embedding vectors were then approximated by their nearest word embeddings, converted back into a token sequence, and fed through the model again. The goal of this experiment was to test the robustness of the model under worst-case conditions where an informed attacker deliberately manipulates the input to fool the classifier.

```

# Map perturbed embeddings back to nearest real words
embedding_weights = model.embedding.weight.detach() # (vocab_size, 100)
perturbed_squeezed = perturbed.squeeze(0).detach() # (seq_len, 100)

# For each token position, find the closest real word
original_tokens = test_texts_tok[sample_idx]
seq_len = min(len(original_tokens), MAX_LEN)

changes = []
for pos in range(seq_len):
    old_id = original_tokens[pos]
    old_word = id_to_word.get(old_id, "<unk>")

    # Find nearest word to the perturbed vector
    dists = torch.cdist(perturbed_squeezed[pos].unsqueeze(0),
embedding_weights).squeeze(0)
    new_id = dists.argmin().item()
    new_word = id_to_word.get(new_id, "<unk>")

    if old_id != new_id:
        changes.append((pos, old_word, new_word))

```

Figure 51: Code to approximate the noised embedding to words

We chose a white-box approach since it provides a transparent stress test of model

vulnerability and highlights whether the classifier relies too heavily on fragile input features. One limitation is that it assumes unrealistic attacker access since external users typically do not know the exact model weights or gradients. Another limitation would be that the reconstructed adversarial text is likely to be unnatural since the perturbed embeddings are projected back into a relatively small, domain-specific embedding space trained only on our dataset vocabulary.

This is the fake, unseen posting from the test set that was chosen as the initial sample.

```
Sample index: 45
P(fake) = 0.9133
ORIGINAL TEXT:
"nurse shift program supervisor us ky lexington apply using below
linklink nurse shift program supervisor on one of five nursing units at
thomson hood veterans center , a state owned long term , 285 bed ,
nursing care facility for kentucky s veterans and is located in
jessamine county . you may contact <unk> <unk> at 1 link submit
applications online at link apply to req <unk> br thomson hood veterans
center does not discriminate against any person on the basis of race ,
color , national origin , disability , or age in admission , treatment ,
or participation in its programs , services , and activities , or in
employment . for further information about this policy , contact amy
lambert , telephone number tdd full time hospital health care"
```

Figure 52: Model evaluation of a Fake Job Sample

After perturbation and mapping to the closest word embeddings in the vector space, the reassembled sequence was then fed into the model as input and it successfully fools the model into misclassifying the perturbed fake sample as a legitimate job posting.

```
Sample index: 45
P(fake) = 0.1012
PERTURBED TEXT:
"bowes bowes bowes bowes bowes bowes bowes bowes bowes bowes bowes bowes
bowes bowes bowes bowes bowes bowes bowes bowes bowes bowes bowes bowes
pitney bowes bowes bowes bowes bowes bowes bowes bowes bowes bowes bowes
bowes bowes bowes bowes bowes companies pitney bowes bowes bowes bowes
bowes companies bowes bowes bowes bowes bowes bowes bowes bowes bowes
bowes bowes bowes bowes bowes bowes bowes bowes bowes bowes bowes bowes
bowes bowes bowes companies bowes bowes bowes bowes bowes bowes bowes
bowes bowes bowes bowes bowes bowes companies bowes bowes bowes bowes
bowes bowes bowes bowes bowes bowes bowes bowes bowes bowes bowes bowes
bowes bowes bowes bowes bowes bowes bowes bowes bowes bowes bowes bowes
companies bowes bowes bowes bowes bowes bowes bowes bowes bowes bowes
bowes bowes bowes companies companies"
```

Figure 53: Model evaluation of a Fake Job Sample that has been perturbed